



Research Intake 2026

Day 38

Introduction to Vision Transformers (ViT)

A Beginner's Guide for Students

Module: Vision Transformers & Vision-Language Models

Learning Computer Vision Through Images, Videos, and Artificial Intelligence

Gudsky Research Foundation

Table of Contents

1	From CNNs to Transformers: Closing the Loop.....	5
1.1	The CNN Era: A Ten-Day Recap.....	6
1.2	The 2017 Origin: Attention for Language, Not Vision	6
1.3	Why Vision Eventually Followed Language's Lead	7
1.4	This Module's Five-Day Roadmap.....	7
1.5	This Day's Practical Project.....	8
1.6	A Note on Terminology	8
2	The Transformer Architecture Recap	9
2.1	Tokens, Embeddings, and Sequences.....	9
2.2	Self-Attention: The Core Mechanism	9
2.2.1	Worked Example: Computing Attention by Hand	10
2.3	Multi-Head Attention	12
2.4	Scaled Dot-Product Attention: Why the Scaling Factor.....	12
2.5	Feed-Forward Layers, Layer Norm, and Residual Connections	13
2.6	Encoder vs. Decoder	14
2.7	Computational Complexity Recap	14
2.8	A Forward Note: Bidirectional vs. Causal Attention	15
2.9	Why Attention Displaced Recurrent Architectures.....	15
3	Patch Embedding: Images as Sequences	16
3.1	The Core Insight: Fixed-Size Patches as Tokens	16
3.2	The Linear Projection Mechanics.....	17
3.3	Contrast With Day 29 §2's Convolution Operation.....	17
3.4	Patch Size as a Design Choice	18
3.4.1	Patch Embedding Walkthrough.....	18
3.5	Overlapping vs. Non-Overlapping Patches	19
4	Position Embeddings for Spatial Awareness	20
4.1	Why Position Information Must Be Added Explicitly	20
4.2	Learned vs. Fixed Sinusoidal Position Embeddings	21
4.3	The [CLS] Token Convention.....	22
4.4	Toward 2D-Aware Position Schemes	24
4.5	Relative Position Encoding: A Closer Look	24
5	The ViT Encoder & Classification Head	25
5.1	Stacking Transformer Encoder Blocks.....	25
5.2	The Classification Head	25
5.3	Parameter Count: ViT vs. a Comparably-Sized CNN.....	26
5.4	The Named Configuration Family	26
5.5	A Complete Forward Pass, Traced End to End.....	28
5.6	Memory and Compute Footprint at Inference.....	28

6	ViT vs. CNN: Inductive Bias Revisited.....	29
6.1	What "Inductive Bias" Means Here, Precisely.....	29
6.2	Why This Makes ViT More Data-Hungry.....	30
6.3	The Practical Crossover Point.....	31
6.4	Beyond Locality: Compositionality and Equivariance.....	31
6.5	The Cost Side of the Ledger.....	32
6.6	A Connection to Few-Shot and Meta-Learning.....	33
7	Training ViT: Data Requirements & Tricks.....	33
7.1	The Original Approach: Massive-Scale Pretraining.....	34
7.2	Strong Data Augmentation.....	34
7.3	Regularization.....	35
7.4	Knowledge Distillation From a CNN Teacher: DeiT.....	35
7.5	Transfer Learning: The Practical Default.....	35
7.6	Optimizer Choices and Learning Rate Schedules.....	36
7.7	Mixed-Precision Training Considerations.....	37
8	ViT in Practice.....	38
8.1	Using Pretrained ViT via Hugging Face and timm.....	38
8.2	The Fine-Tuning Workflow.....	39
8.3	Position Embedding Interpolation at a Different Resolution.....	39
8.4	Common Pitfalls in Practice.....	39
8.5	Visualizing Attention Maps in Practice.....	40
8.6	Debugging With Small Toy Examples.....	41
8.7	Batch Size and Gradient Accumulation.....	42
9	Real-World Applications.....	42
9.1	ViT as a Backbone in Detection and Segmentation.....	43
9.2	Image Classification at Scale.....	44
9.3	ViT as the Visual Encoder Inside Vision-Language Models.....	44
9.4	Medical Imaging, Satellite Imagery, and Video.....	44
9.5	Ethical Considerations: Attention, Trust, and Surveillance at Scale.....	46
10	Research Frontiers.....	47
10.1	Efficient ViT Variants for Edge Deployment.....	47
10.2	Hierarchical and Windowed Attention.....	47
10.3	Scaling Laws for Vision Transformers.....	48
10.4	Forward References.....	48
10.5	Closing Perspective.....	49
10.6	Choosing Between CNN and ViT Today: A Practical Framework.....	49
10.7	Open Research Questions.....	50
11	Common Pitfalls: A Practical Debugging Checklist.....	50
	Glossary of Terms.....	54
	Chapter Summary: Key Takeaways.....	55
	Assignment / Practical Project Brief.....	56
	A Core Requirements.....	56
	B Dataset.....	57
	C Deliverables.....	57

D Grading Rubric	58
E Tips and Common Pitfalls	59
F Submission Checklist.....	60
G Extension Ideas (Optional).....	60

The future of AI is bright, and you can be part of it!

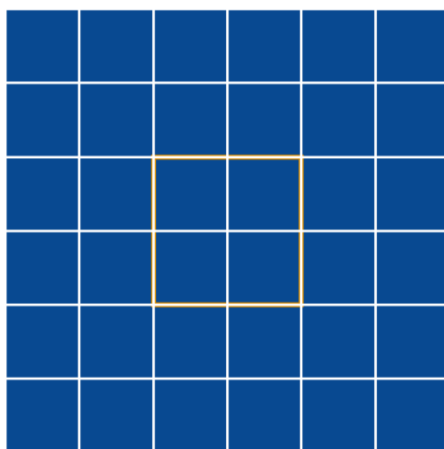
1 From CNNs to Transformers: Closing the Loop

This day opens a new module, and it is worth being explicit about exactly what is changing and what is not. Days 26 through 37 developed the Computer Vision Fundamentals module in full — an arc that assumed, from Day 29 onward, that convolution was the core computational operation any vision architecture would be built from. Day 30 §7 and Day 33 §8 both introduced an alternative to that assumption — Vision Transformers — but deliberately as a supporting concept, previewed just enough to make sense of a specific comparison (Day 30 §9's backbone-selection framework) or a specific application (Day 33 §8.2's SegFormer and SAM). This day, and the four days following it, ask the question those earlier previews set up but did not answer: what happens if convolution is replaced with attention entirely, as the core operation an entire vision architecture is built from?

This is a genuine architectural departure, not an incremental refinement of the kind Day 30 traced across VGG, ResNet, and EfficientNet. Every one of those architectures, however different in detail, shared convolution's defining structural assumption: a unit's receptive field is local, and grows only gradually with network depth (Day 29 §3.3). A Vision Transformer abandons that assumption outright. This day develops, from first principles and in considerably more depth than the earlier previews allowed, exactly how that abandonment works mechanically (Sections 2 through 5), what it costs and what it buys (Section 6), and how to work with it in practice (Sections 7 through 9).

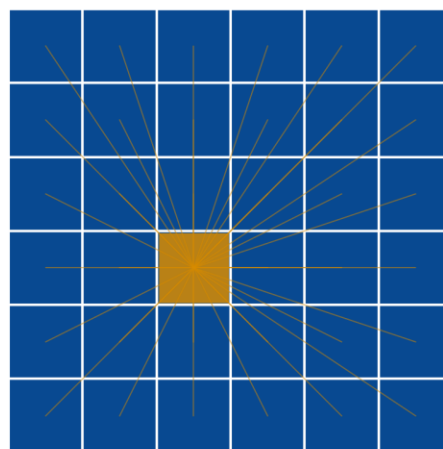
Callout Box: *A one-line intuition worth holding onto through the more technical sections that follow: a CNN looks at an image through a sliding local window, one small neighborhood at a time, building up a global picture only gradually across many layers (Day 29 §3.3). A ViT looks at the whole image at once from its very first layer, and lets self-attention decide which parts of the image are relevant to which other parts. Figure 1 makes this distinction directly visible.*

CNN: A Sliding Local Window



a kernel sees only a small local neighborhood at each layer (Day 29 §2.4)

ViT: Global Attention From Layer 1



self-attention relates every patch to every other patch directly (§2)

Figure 1. The central architectural distinction this day develops: a CNN's kernel (left) sees only a small local neighborhood, with its effective receptive field growing gradually across many stacked layers (Day 29 §3.3). A ViT's self-attention (right) relates every patch directly to every other patch from the very first layer — no depth-dependent growth required at all.

1.1 The CNN Era: A Ten-Day Recap

It is worth briefly retracing the specific arc this day departs from, since the departure is easier to appreciate with the destination clearly in view. Day 29 established the convolution operation from first principles — local receptive fields, weight sharing, translation equivariance — and the classic conv-pool-flatten-dense architectural template. Day 30 traced a decade of refinement to that template: VGG's disciplined depth, ResNet's residual connections solving the vanishing-gradient problem at extreme depth, Inception's multi-scale parallel branches, and MobileNet/EfficientNet's efficiency-focused redesigns. Days 31 through 33 built detection, tracking, and segmentation systems on top of these backbones. Days 34 and 35 turned the same convolutional machinery toward generation rather than discrimination. Day 36 extended it into 3D and video. Day 37 closed the module with the explainability tools needed to trust a convolutional model's decisions in a high-stakes domain. Every one of these eleven days shared one assumption in common — and this day is the first to set that assumption aside entirely.

1.2 The 2017 Origin: Attention for Language, Not Vision

The transformer architecture, developed in full in Section 2, did not originate in computer vision at all. It was introduced in 2017 for machine translation — a natural language processing task requiring a model to relate words across a sentence regardless of how far apart they appear ("the trophy didn't fit in the suitcase because it was too big" requires resolving what "it" refers to, potentially many words away). Self-attention (Section 2.2) solved this relating-distant-elements problem directly, and did so more effectively, and with

more parallelizable training, than the recurrent architectures that had dominated NLP until that point. For three years, this remained a language-specific architecture, with no obvious or agreed-upon path into computer vision at all.

1.3 Why Vision Eventually Followed Language's Lead

The 2020 Vision Transformer paper, developed in full starting in Section 3, closed that gap with a deceptively simple insight: if an image could be converted into a sequence of tokens — the same input format a transformer already expected — then the entire architecture, self-attention and all, could be applied to vision with no fundamental redesign at all. This is worth appreciating as a genuinely elegant piece of research: rather than inventing a vision-specific attention mechanism from scratch, ViT's contribution was almost entirely in the input representation (Section 3's patch embedding), leaving the proven transformer encoder itself essentially untouched. The architecture's eventual success in vision was not a foregone conclusion at the time — Section 6 develops in detail why a transformer's lack of any vision-specific structural assumption made this initially a genuinely uncertain bet, one that paid off only once sufficient training data was available to compensate.

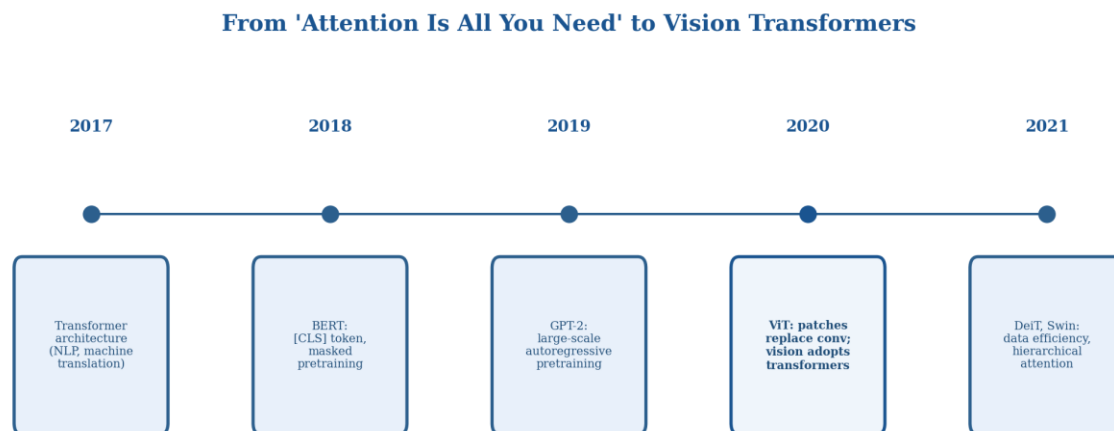


Figure 11. Five years separate the transformer's original 2017 introduction for machine translation and ViT's 2020 adaptation to vision — with BERT's [CLS] token convention (§4.3) and GPT-2's demonstration of large-scale pretraining both landing in between, and both directly informing ViT's own design choices.

1.4 This Module's Five-Day Roadmap

This day is the first of a five-day module: ViT foundations (Day 38, this day) establish the architecture itself; advanced ViT variants (Day 39) address the practical limitations this day's Section 6 and Section 10 identify; extending to language (Day 40) begins combining visual and textual understanding; CLIP's contrastive approach (Day 41) develops a specific, highly influential vision-language training method; and modern vision-language models and deployment (Day 42) close the module with today's production systems. Every one of these later days assumes fluent command of this day's content — the patch embedding, position embedding, and transformer encoder mechanics developed in Sections 2 through 5 are

the direct technical prerequisite for everything that follows, in exactly the sense Day 29's CNN fundamentals were the prerequisite for Day 30's architecture survey.

Day	Topic	Direct Dependency on This Day
38 (this day)	ViT foundations	—
39	Advanced ViT architectures	Windowed/hierarchical attention directly extends §2's self-attention and §10.2's preview
40	Extending to language	Uses ViT (§3–5) as the visual half of a joint vision-language system
41	CLIP's contrastive approach	Trains a ViT-based image encoder jointly with a text encoder
42	Modern VLMs and deployment	Assumes fluency with every mechanism this day develops

1.5 This Day's Practical Project

This day carries a graded assignment, PP16, developed in full at the end of this document. PP16 asks for two complete implementations side by side — a Vision Transformer built entirely from first principles (Task A) and a pretrained Vision Transformer fine-tuned via transfer learning (Task B) — specifically so that Section 6's inductive-bias argument can be verified empirically on your own results, not merely taken on faith from this day's prose. Sections 2 through 5 are the direct technical prerequisite for Task A; Sections 7 and 8 are the direct technical prerequisite for Task B.

1.6 A Note on Terminology

Three closely related terms recur throughout this day and are worth disambiguating explicitly before proceeding, since casual usage in the broader field sometimes blurs them together. "Transformer" refers to the general architecture family developed in Section 2 — self-attention, multi-head attention, feed-forward layers, residual connections — originally designed for and still heavily used in natural language processing, with no inherent connection to vision at all. "Attention" (or "self-attention") refers specifically to the core weighting mechanism within a transformer (Section 2.2) and is sometimes used loosely to refer to the transformer architecture as a whole, though the two are not strictly synonymous — a model can incorporate an attention mechanism without being a full transformer, and Day 39 §4's efficient attention variants sometimes modify or approximate the attention mechanism specifically while retaining the broader transformer structure around it. "Vision Transformer" (ViT) refers specifically to this day's architecture — a transformer, in the general sense, applied to images via the patch embedding scheme Section 3 develops. Every ViT uses a transformer and uses attention; not every transformer or every use of attention is a ViT. Keeping this hierarchy straight — transformer as the general architecture, attention as its core mechanism, ViT as its specific vision application — is worth doing deliberately, since Day 39 through Day 42's content

will introduce further, more specific named variants and applications built on this same three-level hierarchy.

2 The Transformer Architecture Recap

Before Section 3 can develop how an image becomes a sequence a transformer can process, it is necessary to recap the transformer architecture itself in genuine mechanical detail — not merely a passing gesture, since Sections 5 through 8 all depend on a precise understanding of exactly how self-attention computes what it computes. This section is more thorough than a typical "recap" might suggest, deliberately, because the remainder of this day builds directly and repeatedly on every mechanism developed here.

2.1 Tokens, Embeddings, and Sequences

A transformer operates on a sequence of vectors, each called a token embedding. In natural language processing, a token is typically a word or sub-word unit, and its embedding is a learned vector representation of that unit's meaning — the same general notion of a learned vector representation Day 29 §9.1 used to describe a CNN filter's response, now applied to a discrete input unit rather than a spatial image region. A sequence of N tokens is represented as an $N \times d$ matrix, where d is the embedding dimension — every position in the sequence is a d -dimensional vector, and the entire transformer architecture operates on this matrix, transforming it layer by layer while preserving its $N \times d$ shape throughout (a property Section 5.1 returns to directly).

2.2 Self-Attention: The Core Mechanism

Self-attention computes, for every element in an input sequence, a weighted combination of every other element, where the weights are themselves computed from the sequence's own content — hence "self"-attention, as distinct from an attention mechanism relating two different sequences (cross-attention, which Day 35 §7.2 already introduced in the context of diffusion models attending to text conditioning). Concretely, each input token embedding is projected, via three separate learned linear layers, into a query vector, a key vector, and a value vector. Figure 8 lays out this projection step directly.

Computing Queries, Keys, and Values

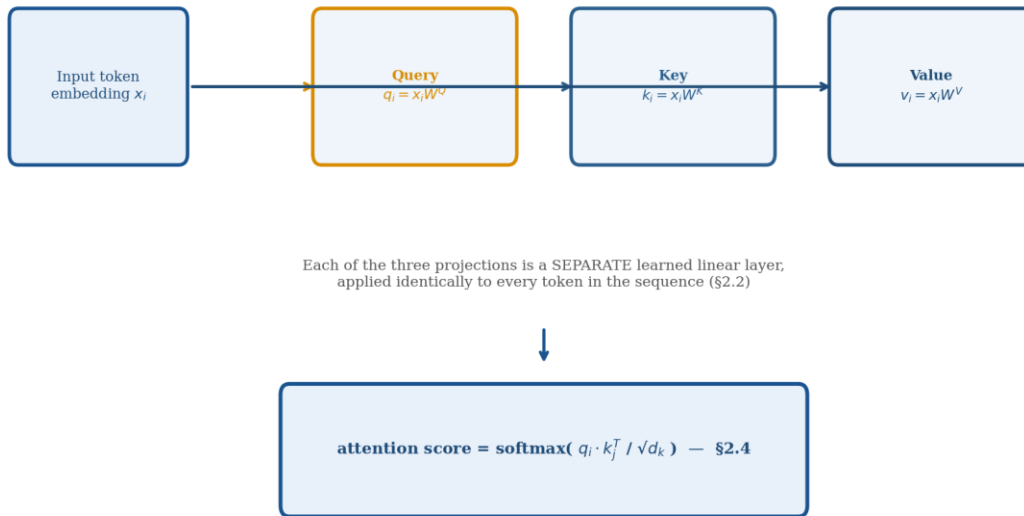


Figure 8. Every token embedding is projected through three separate learned linear layers into a query, a key, and a value vector. The query and key vectors are compared (via a scaled dot product) to compute attention scores; the value vectors are what actually get combined, weighted by those scores, into each token's updated representation.

A given token's query is compared against every token's key (via a dot product) to produce a set of raw attention scores, one per token pair; these scores are scaled (Section 2.4 develops exactly why) and normalized via softmax into attention weights that sum to 1 across each row. These weights are then used to combine every token's value vector into that specific token's updated representation. The practical effect: a token's updated representation is a weighted blend of the entire sequence's content, with the weights determined by how relevant the mechanism judged each other token to be — precisely the global, content-dependent relevance computation Figure 1's right panel visualized directly, now made mathematically precise.

2.2.1 Worked Example: Computing Attention by Hand

Consider a toy sequence of 6 tokens (this could be 6 word tokens, or — anticipating Section 3 — 6 image patches), each already projected into query, key, and value vectors of some dimension d_k . Figure 10's left panel shows a concrete, if synthetic, set of raw attention scores for this 6-token sequence — the result of computing $q_i \cdot k_j^T / \sqrt{d_k}$ for every pair (i, j) . Notice that these raw scores are unbounded real numbers, both positive and negative, with no immediately interpretable scale.

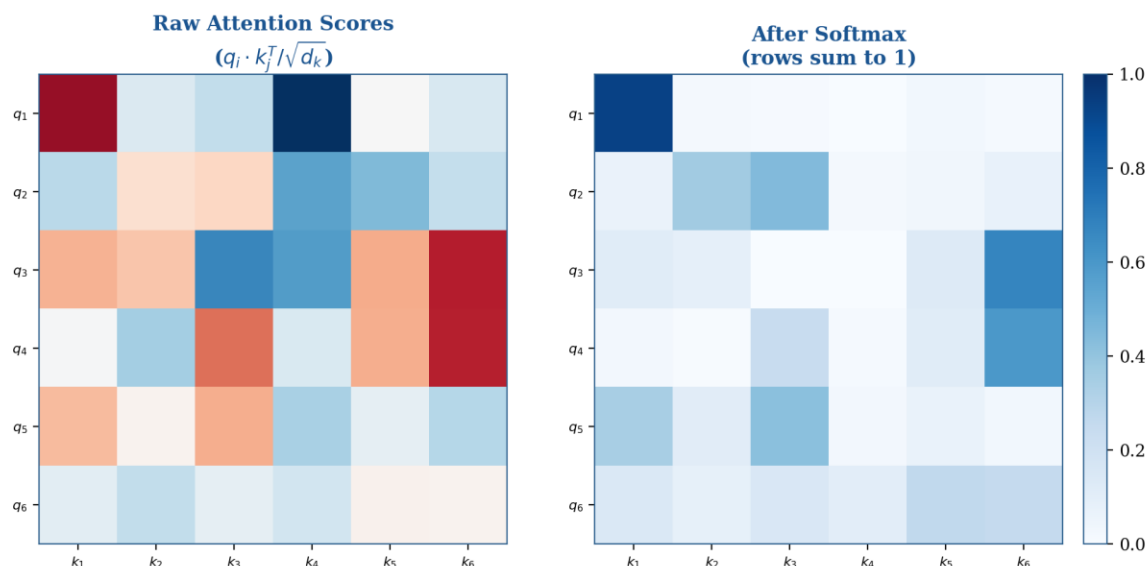


Figure 10. A worked, if synthetic, example of the attention computation's middle two steps. Left: raw attention scores, one per query-key pair, computed as a scaled dot product — unbounded, both positive and negative. Right: the same scores after row-wise softmax, now interpretable as a proper probability distribution — for query sq_1 , notice that the weight on key sk_1 dominates almost completely (a strong, confident match), while for sq_3 and sq_4 , weight is split more evenly between sk_1 and sk_6 (a genuinely ambiguous relevance judgment).

Applying softmax independently to each row converts these raw scores into a proper probability distribution over the other tokens — every row now sums to exactly 1, and every individual weight lies between 0 and 1, directly interpretable as "how much of token j 's information should flow into token i 's updated representation." Figure 10's right panel shows the result: query 1's attention is heavily concentrated on key 1 (a confident, nearly one-hot relevance judgment), while query 3 and query 4 split their attention more evenly across several keys — a genuinely ambiguous case where several tokens seem comparably relevant. This variability — sometimes confident and sharp, sometimes diffuse and uncertain — is not a flaw; it is exactly the flexible, content-dependent relevance computation self-attention is designed to provide, and it is what a fixed, hand-designed convolution kernel (Day 29 §2.1) simply cannot represent at all, since a kernel's weighting of its neighborhood is fixed after training and identical for every input, not recomputed per input as self-attention's weights are.

The final step multiplies these softmax weights against the sequence's value vectors and sums them: for query 1, whose weight is concentrated almost entirely on key 1, the updated representation will closely resemble token 1's own value vector; for query 3, whose weight splits between several keys, the updated representation will be a genuine blend of several tokens' value vectors. This entire three-step process — score, scale-and-softmax, weighted-sum — is self-attention's complete mechanical definition, and it is worth having this concrete worked example in mind for every later section that refers back to "the attention computation."

2.3 Multi-Head Attention

Rather than computing a single attention pattern, multi-head attention runs several independent attention computations ("heads") in parallel, each with its own learned query, key, and value projections, and concatenates their outputs before a final linear projection. Figure 9 shows this parallel structure directly.

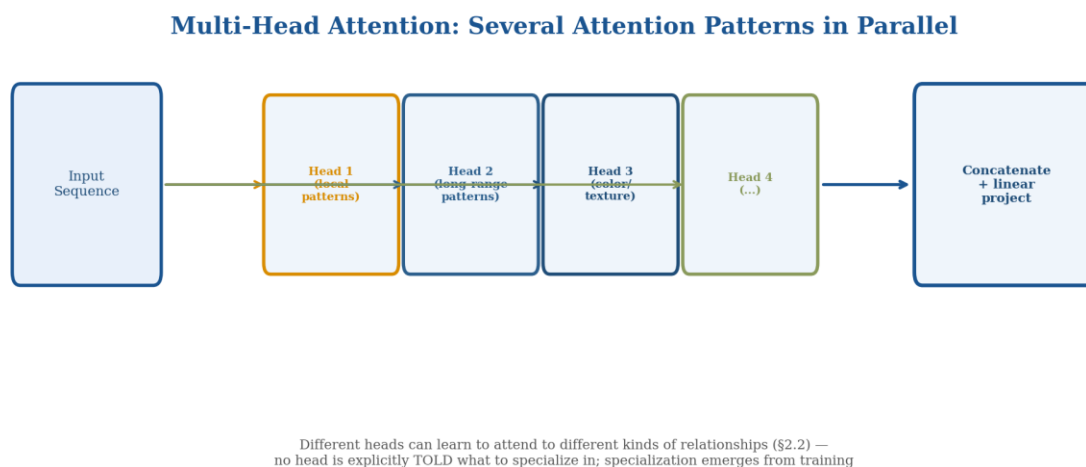


Figure 9. Multi-head attention runs several independent attention computations in parallel, each with its own learned $Q/K/V$ projections, then concatenates the results and applies one final linear projection. Different heads can, empirically, learn to specialize in different kinds of relationships — no head is explicitly told what to attend to; specialization emerges purely from training.

Different heads can, in practice, learn to attend to different kinds of relationships — one head might learn to attend primarily to nearby elements (a learned approximation of the locality a CNN gets for free, per Section 6.1's argument), another to elements sharing some more abstract property unrelated to spatial or sequential distance at all. Giving the model several of these specialized "views" of the sequence, computed in parallel and then combined, produces a richer, multi-faceted relevance computation than a single attention pattern could represent alone — the same divide-and-specialize logic Day 30 §5.1's Inception module applied to convolutional kernel sizes, now recurring for attention patterns instead.

2.4 Scaled Dot-Product Attention: Why the Scaling Factor

It is worth pausing specifically on the $\sqrt{d_k}$ scaling factor introduced without full justification in Section 2.2, since it addresses a genuine, easily-overlooked numerical problem. As the query and key vectors' dimension d_k grows, the dot product $q_i \cdot k_j^T$ tends to grow in magnitude as well (summing more individual products), and very large dot-product values push the subsequent softmax into a regime where its gradient becomes vanishingly small — a specific instance of the same saturating-activation vanishing-gradient problem Day 29 §5.2 developed for sigmoid and tanh activations, now arising from an unbounded input magnitude rather than from the activation function's own shape. Dividing every raw score by $\sqrt{d_k}$ keeps the dot product's expected magnitude roughly constant regardless of d_k , keeping softmax in a numerically well-behaved regime throughout training — a small, easily-missed detail whose absence can

otherwise silently degrade a transformer's trainability, particularly at the larger embedding dimensions Section 5.4's larger configurations use.

2.5 Feed-Forward Layers, Layer Normalization, and Residual Connections

Each transformer encoder block combines multi-head attention (Sections 2.2–2.4) with a position-wise feed-forward network (a small MLP applied identically and independently to every sequence position, typically expanding to a larger intermediate dimension and back — commonly $4\times$ the embedding dimension), layer normalization (normalizing each individual sequence element's representation across its own feature dimension, distinct from Day 29 §8.3's batch normalization, which normalizes across a batch of examples instead), and residual connections around both the attention and feed-forward sub-layers. Figure 2 lays out the complete block.

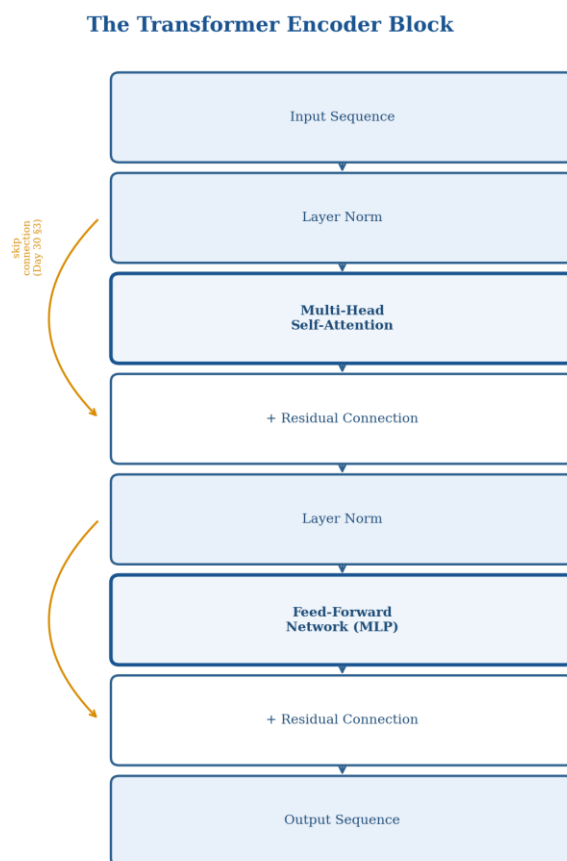


Figure 2. A complete transformer encoder block: layer normalization, multi-head self-attention, and a residual connection, followed by a second layer normalization, a feed-forward network, and a second residual connection. Stacking many of these blocks (§5) is what gives a transformer its depth.

The residual connections are worth pausing on specifically, since they are not a cosmetic detail. Day 30 §3.3–§3.4 developed, at length, why residual connections were necessary to make extreme CNN depth trainable at all — an unimpeded additive path for gradients to flow backward through, regardless of how

poorly-conditioned gradient flow through the intervening layers happens to be. Transformer encoder blocks use exactly the same mechanism, for exactly the same reason: a Vision Transformer with many stacked encoder blocks (Section 5) would face the same vanishing-gradient risk Day 30 §3.2 developed for plain CNNs, were it not for this residual structure — the identical architectural fix recurring in a structurally different network family.

Layer normalization's placement is itself a genuine design choice worth naming: the "pre-norm" configuration (normalizing before each sub-layer, as Figure 2 shows) has become the standard default in most modern transformer implementations, including ViT, since it empirically produces more stable training than the original transformer paper's "post-norm" configuration (normalizing after each sub-layer and its residual addition) — a refinement discovered through subsequent research after the original 2017 architecture, worth knowing about specifically because implementations copying the original paper's exact ordering can behave measurably differently in practice.

2.6 Encoder vs. Decoder: Why ViT Uses Only the Encoder Half

The original transformer architecture, developed for machine translation, combined an encoder (processing the full input sequence, exactly as Sections 2.1 through 2.5 developed) with a decoder (generating an output sequence one element at a time, attending back to the encoder's output via cross-attention). A Vision Transformer, per Section 5, uses only the encoder half: an image classification task needs to process a fixed input (the patch sequence) and produce a single output (a class label), with no sequential generation step required — the same reason Day 33 §3.1's Fully Convolutional Networks needed no decoder-style sequential generation either, despite the superficial naming overlap with Day 33 §3.2's own, differently-purposed encoder-decoder terminology for dense prediction. It is worth flagging this terminology collision explicitly: Day 33's "encoder-decoder" describes a downsampling-then-upsampling spatial structure; the transformer's "encoder-decoder" describes a full-sequence-processing-then-sequential-generation structure — related in name only, not in mechanism.

2.7 Computational Complexity Recap

Self-attention's computational cost is worth stating precisely here, since Section 3.4.1 and Day 39 §2 both depend on it directly. Computing attention scores between every pair of N tokens costs $O(N^2 \times d)$ — quadratic in sequence length N , linear in embedding dimension d . This stands in direct contrast to a convolutional layer's cost, which scales linearly with the number of spatial positions (since a kernel's computation at each position does not depend on every other position at all, per Day 29 §2.4's local receptive field). This quadratic-versus-linear distinction is the single most consequential practical difference between the two architecture families, and it is the exact mechanism Section 3.4.1's worked example quantifies concretely for a specific patch-size choice, and that Day 39 §2 develops as this day's immediate successor's central technical problem.

2.8 A Forward Note: Bidirectional vs. Causal Attention

One further distinction is worth introducing here, briefly, ahead of its full relevance in Day 40 onward's language-modeling content. Section 2.2's self-attention, as developed for ViT, is bidirectional: every token can attend to every other token in the sequence, regardless of position — appropriate for an image, where there is no meaningful "earlier" or "later" patch, only a 2D spatial arrangement. Section 4 restores via position embeddings. Autoregressive language models (previewed briefly here since Day 40 onward will need the distinction) instead use causal attention: a token may only attend to itself and earlier tokens in the sequence, never later ones, since the model must generate text one token at a time without access to tokens it has not yet produced. This distinction matters directly for Day 40's vision-language systems, which frequently combine a bidirectional ViT-style visual encoder (this day's architecture, processing a fixed, fully-available image) with a causal, autoregressive text decoder (generating a caption or response one word at a time) — two different attention masking conventions coexisting within a single joint system, each appropriate to its own half's actual information-availability structure.

2.9 Why Attention Displaced Recurrent Architectures

It is worth developing Section 1.2's brief historical mention slightly further, since the specific reason attention displaced the recurrent architectures that preceded it in NLP is directly illuminating for understanding self-attention's own structural properties. A recurrent neural network processes a sequence one element at a time, in strict order, with each step's computation depending on the previous step's hidden state — a structure that makes relating two distant sequence elements slow (information must propagate step by step across every intervening position) and, more practically, makes training difficult to parallelize (step t 's computation cannot begin until step $t-1$'s computation has finished, since it depends on that step's output). Self-attention's $O(N^2)$ cost (Section 2.7) is, in an important practical sense, a trade deliberately accepted in exchange for removing this sequential dependency entirely: every token's attention computation depends only on the full input sequence being available simultaneously, not on any other token's computation having already finished, meaning the entire computation for every token can proceed in parallel on modern GPU hardware. This parallelization advantage — not superior sequence-length scaling, which is actually worse than a recurrent network's linear cost — is a large part of what made the transformer architecture practically trainable at the scale Section 7.1's massive-scale pretraining requires, and it is worth remembering this trade explicitly the next time Section 2.7's quadratic cost is discussed as a pure drawback: it replaced a different, in some ways more limiting, computational bottleneck rather than introducing a cost where none existed before.

Property	Recurrent (RNN/LSTM)	Self-Attention (§2.2)
Sequential dependency	Yes — step t needs step $t-1$'s output	No — all positions computed simultaneously

Property	Recurrent (RNN/LSTM)	Self-Attention (§2.2)
Parallelizable training?	Limited, along the sequence dimension	Highly parallelizable
Cost to relate two distant elements	Scales with their distance apart	Constant — a single attention computation
Overall computational cost	Linear in sequence length	Quadratic in sequence length (§2.7)

3 Patch Embedding: Images as Sequences

Section 2's self-attention mechanism operates on a sequence of vectors — exactly the input format an NLP transformer's word tokens naturally provide, and exactly the input format a 2D image does not. This section develops ViT's foundational insight in full: a specific, simple procedure for converting an image into a sequence a transformer encoder can process directly, together with the design tradeoffs that specific procedure introduces.

3.1 The Core Insight: Fixed-Size Patches as Tokens

An input image is divided into a grid of fixed-size, non-overlapping patches — commonly 16×16 pixels, though the specific size is a tunable design choice developed further in Section 3.4's worked example and revisited in Day 39 §1's discussion of its limitations. Each patch is flattened into a single vector (a $16 \times 16 \times 3$ patch, for a standard RGB image, flattens to a 768-dimensional vector) and linearly projected into the transformer's working embedding dimension. The resulting sequence of patch embeddings is then treated exactly like a sequence of word token embeddings in an NLP transformer — self-attention (Section 2.2) has no intrinsic notion of "this token came from an image patch" versus "this token came from a word"; it operates identically on either kind of input sequence, since ViT's entire architectural insight lies in this initial patch-tokenization step, not in any change to the transformer encoder itself.

Patch Embedding: An Image Becomes a Sequence

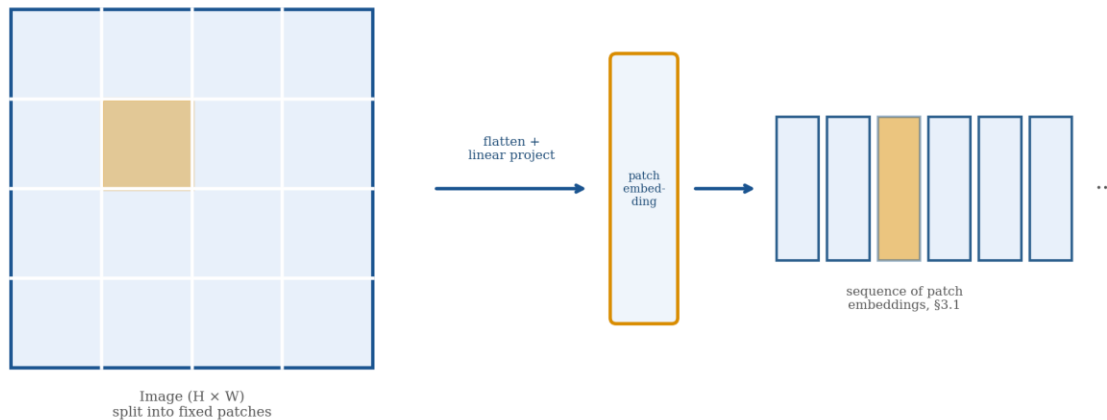


Figure 3. Patch embedding: an image is divided into a grid of fixed-size patches, each patch is flattened and linearly projected into the transformer's embedding dimension, producing a sequence of patch embeddings — the direct input format Section 2's self-attention mechanism requires.

3.2 The Linear Projection Mechanics

It is worth being precise about what "linearly projected" means mechanically, since it is easy to gloss over. Flattening a $16 \times 16 \times 3$ patch produces a 768-dimensional vector; this vector is multiplied by a learned weight matrix of shape $768 \times d$ (where d is the transformer's working embedding dimension, commonly also 768 for ViT-Base — a coincidence of the standard configuration, not a mathematical necessity) to produce the final patch embedding. Critically, this projection matrix is shared identically across every patch in every image — the same learned weights process the top-left patch and the bottom-right patch alike, echoing Day 29 §2.4's weight-sharing principle even though the surrounding computation (self-attention rather than convolution) is entirely different. A common and illuminating implementation detail: this entire flatten-and-project step can be implemented as a single strided convolution — a convolutional layer with kernel size and stride both equal to the patch size, and output channel count equal to d — producing numerically identical results to an explicit flatten-then-matrix-multiply, purely because a non-overlapping patch convolution and a per-patch linear projection are mathematically the same operation. This is a genuinely interesting bridge between the two architecture families worth pausing on: the very first operation inside a "pure" Vision Transformer is, mechanically, a single convolutional layer — just one with an unusually large kernel and stride, applied exactly once, with no sliding-window reuse of intermediate results the way a standard CNN's many-layer convolutional stack provides.

3.3 Contrast With Day 29 §2's Convolution Operation

It is worth being precise about how patch embedding differs from a convolutional layer's own sliding-window processing, since both operations initially look at small local regions of an image. A convolutional kernel (Day 29 §2.1) slides across the entire image, applying the identical weights at every position (weight

sharing, Day 29 §2.4), and produces a new, still-spatially-organized feature map — the output remains an image-shaped grid, just with different channel depth. Patch embedding instead divides the image into non-overlapping regions once, and flattens each into an independent sequence element with no further spatial sliding at all — the output is a sequence, not a spatially-organized grid, and this sequence's elements have no inherent 2D adjacency relationship a subsequent transformer layer could exploit automatically, in sharp contrast to how a subsequent convolutional layer automatically inherits its input's 2D spatial structure. This loss of automatic spatial structure is precisely what Section 4's position embeddings exist to restore.

Property	Convolution (Day 29 §2)	Patch Embedding (§3.1–3.2)
Output structure	Spatially-organized feature map	Flat sequence of tokens
Weight reuse	Slides across the whole image, many positions	Applied once per patch, no sliding
Spatial adjacency after the operation	Automatic — grid structure preserved	Lost — must be restored explicitly (§4)
Receptive field of one output unit	Local, grows with depth (Day 29 §3.3)	One full patch, immediately

3.4 Patch Size as a Design Choice

Patch size is not a fixed architectural constant but a tunable hyperparameter with real, quantifiable consequences developed in the worked example immediately below. Smaller patches preserve finer spatial detail (each token corresponds to a smaller, more spatially precise region of the original image) but produce a longer sequence; larger patches produce a shorter, cheaper-to-process sequence at the cost of coarser spatial granularity — the same fundamental resolution-versus-cost tradeoff Day 27 §2.1's image resizing decisions and Day 30 §6's efficient-architecture design both grappled with, now recurring specifically in the context of Section 2.7's quadratic attention cost.

3.4.1 Patch Embedding Walkthrough

Consider a standard 224×224 RGB input image, divided into 16×16 patches. The number of patches along each spatial dimension is $224 / 16 = 14$, giving a total of $14 \times 14 = 196$ patches across the full image. Each patch, flattened, is a vector of length $16 \times 16 \times 3 = 768$ (patch height × patch width × color channels). This 768-dimensional flattened patch is linearly projected — a single learned matrix multiplication, per Section 3.2 — into the transformer's working embedding dimension (commonly also 768 for the standard "ViT-Base" configuration, though this is a separate, independently tunable design choice from the flattened patch dimension). The result is a sequence of 196 embedding vectors, each of dimension 768 — the direct input Section 5's transformer encoder stack processes.

 Worked Example: Patch Size and Sequence Length

Patch size directly determines sequence length, and this relationship matters a great deal for the quadratic attention cost Section 2.7 developed and Day 39 §2 extends in full: a 224×224 image at 16×16 patches produces 196 tokens; the same image at a smaller 8×8 patch size produces $(224/8)^2 = 784$ tokens — four times as many.

Since self-attention's computational cost scales with the SQUARE of sequence length (§2.7), quadrupling the token count does not merely quadruple compute cost — it increases it by roughly 16x, a direct, quantitative preview of exactly the practical tension Day 39 develops as this day's immediate successor.

Working the arithmetic through explicitly: attention cost scales as $O(N^2)$, so a 4x increase in N produces a $4^2 = 16$ x increase in attention cost specifically (the feed-forward sub-layer's cost, by contrast, scales only linearly with N , so the OVERALL block cost increase is somewhat less than 16x, but attention's quadratic term dominates as N grows large).

This is why patch size is not a free parameter to shrink arbitrarily in pursuit of finer spatial detail — smaller patches capture more fine-grained visual information, per Day 39 §1's callout about a single patch size struggling to represent both small objects and fine texture, but at a computational cost that grows punishingly fast.

A second worked comparison is worth making explicit: at 32×32 patches, the same 224×224 image produces only $(224/32)^2 = 49$ tokens — a quarter of the standard 196-token sequence, and a correspondingly cheaper attention computation ($49^2 = 2,401$ pairwise comparisons, versus $196^2 = 38,416$ at the standard patch size — roughly a sixteenth the cost). This is precisely why ViT variants are sometimes named with their patch size explicitly ("ViT-Base/16" versus "ViT-Base/32"), and why Section 8.1's `google/vit-base-patch16-224` checkpoint name encodes both the patch size and the expected input resolution directly in its identifier — both are load-bearing configuration choices, not incidental naming detail.

3.5 Overlapping vs. Non-Overlapping Patches

ViT's original formulation uses strictly non-overlapping patches, per Section 3.1 — the patch grid tiles the image exactly, with every pixel belonging to precisely one patch. A natural variant, worth flagging as a design space rather than a settled question, uses overlapping patches (a smaller stride than patch size, echoing Day 29 §3.1's stride-versus-kernel-size distinction for convolution) — this increases sequence length for a given patch size, trading Section 3.4's cost increase for potentially smoother spatial coverage at patch boundaries, since a fixed non-overlapping tiling can, in principle, split a small but semantically important visual feature awkwardly across two adjacent patches with no single token capturing it cleanly. This tradeoff has motivated some of the hybrid and convolutional-stem approaches Day 39 §6 develops, which use a small amount of genuine convolution (with its natural overlapping-window behavior) specifically at the patch-embedding stage, before handing off to a standard transformer encoder for the remaining layers.

Callout Box: *A real-world illustration worth connecting back to Section 3.4.1's cost analysis directly: production systems processing very high-resolution images (satellite imagery, per Section 9.4's forward reference, or gigapixel histopathology slides, echoing Day 37 §7.1's modality survey) frequently cannot*

afford a standard 16×16 patch size at all, since the resulting sequence length would make Section 2.7's quadratic attention cost computationally prohibitive. A larger patch size — 32×32 or beyond — is a common, practical response, deliberately trading finer spatial granularity for a tractable sequence length, exactly the same resolution-versus-cost negotiation Day 27 §2.1's image-resizing decisions required for entirely different reasons.

Checkpoint Name	Patch Size	Resolution	Resulting Sequence Length
ViT-Base/32	32×32	224×224	49 tokens (+ [CLS] = 50)
ViT-Base/16	16×16	224×224	196 tokens (+ [CLS] = 197)
ViT-Base/16 (high-res)	16×16	384×384	576 tokens (+ [CLS] = 577)
ViT-Base/8	8×8	224×224	784 tokens (+ [CLS] = 785)

This table is worth reading as a direct, concrete instantiation of Section 3.4.1's cost argument across real, commonly-released checkpoint configurations: moving from ViT-Base/32 to ViT-Base/16 at the same resolution roughly quadruples sequence length (and, per Section 2.7, roughly multiplies attention cost by a much larger factor still); moving to a higher 384×384 resolution at the same 16×16 patch size very nearly triples sequence length again. Every one of these configurations exists as a real, published checkpoint precisely because different applications land at different points along this same resolution-versus-cost tradeoff — worth checking directly, per Section 8.1's naming-convention discussion, before selecting a checkpoint for a specific project's actual latency and accuracy requirements.

4 Position Embeddings for Spatial Awareness

Section 3.3 flagged a specific consequence of patch embedding worth developing in full here: the resulting sequence of patch embeddings has no inherent notion of spatial position at all. This section develops why that is a genuine problem, the two standard fixes, and the [CLS] token convention that completes ViT's input representation.

4.1 Why Position Information Must Be Added Explicitly

Self-attention (Section 2.2) is, by construction, permutation-invariant: shuffling the order of a sequence's elements produces the identical set of pairwise attention computations, just relabeled — the mechanism has no built-in sensitivity to sequence order at all. This is worth proving to yourself directly from Section 2.2's mechanics: the attention score between token i and token j depends only on their query and key vectors, computed independently of where in the sequence either token sits — relabeling every token's position, without changing its actual content, leaves every pairwise score, and therefore every attention weight, completely unchanged. For an NLP transformer, this would mean "the dog bit the man" and "the man bit the dog" would be processed identically without some additional signal distinguishing word order; for a

Vision Transformer, it would mean a patch from the image's top-left corner and a patch from its bottom-right corner would be treated as interchangeable, with no signal at all indicating where in the image each patch actually came from. This is a serious problem for a vision task specifically, since spatial arrangement is frequently central to what an image actually depicts — the same set of patches, differently arranged, can depict a completely different scene.

Callout Box: *A concrete illustration worth holding in mind: take a photograph of a face, cut it into a 4x4 grid of patches, and shuffle those sixteen patches into a random new arrangement. A human viewer immediately recognizes the result as scrambled nonsense, not a face — but per §4.1's argument, self-attention alone, with no position embeddings, would process the scrambled arrangement and the correctly-ordered original completely identically, since the mechanism has no way to represent that the eyes belong above the nose and the nose belongs above the mouth, only that these patches exist SOMEWHERE in the sequence together.*

4.2 Learned vs. Fixed Sinusoidal Position Embeddings

Two standard approaches exist for supplying this missing position information, both by adding (element-wise) a position-specific vector to each patch embedding before it enters the transformer encoder. Fixed sinusoidal position embeddings use a predetermined mathematical function (sine and cosine waves at varying frequencies, a function of each position's index) requiring no training at all — the original transformer architecture's own approach. Learned position embeddings instead treat each position's embedding as a free parameter, trained via backpropagation exactly like every other weight in the network, and initialized randomly before training begins. ViT's original formulation uses learned position embeddings, which is why Section 8.3's fine-tuning-at-different-resolution discussion involves an interpolation step specifically for these learned embeddings — a fixed sinusoidal scheme, by contrast, can simply be recomputed at any sequence length with no interpolation required, a genuine practical advantage learned embeddings give up in exchange for greater representational flexibility.

Figure 12 visualizes the fixed sinusoidal scheme directly, since its specific mathematical structure is worth seeing rather than only stating abstractly. Each embedding dimension corresponds to a sine or cosine wave at a different frequency — low dimensions oscillate rapidly across position, high dimensions oscillate slowly — and a given position's full embedding vector is the specific combination of all these waves' values at that position. This specific construction has an elegant mathematical property worth naming: the embedding for any position can be expressed as a fixed linear function of the embedding for any other position, meaning relative position information (how far apart two tokens are) is implicitly recoverable from the embeddings themselves, not just absolute position — a property some learned-position-embedding schemes must instead approximate purely through training rather than receiving for free from the construction.

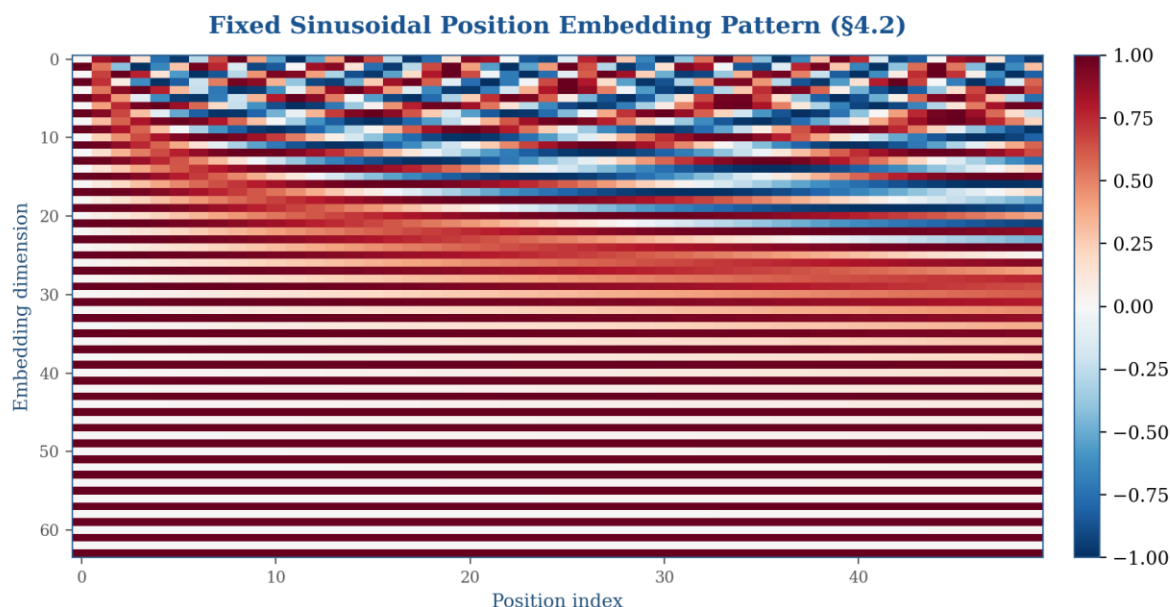


Figure 12. The fixed sinusoidal position embedding pattern: each embedding dimension corresponds to a sine or cosine wave at a distinct frequency, with low dimensions varying rapidly across position and high dimensions varying slowly. A given position's full embedding is this pattern's specific vertical slice — visually, no two positions produce an identical pattern, which is exactly the property needed to make positions distinguishable.

Research Spotlight: Do Learned Position Embeddings Actually Learn Spatial Structure?

A natural empirical question, directly testable and frequently tested in ViT research: once trained, do a Vision Transformer's learned position embeddings actually encode something resembling 2D spatial structure, despite receiving no explicit 2D grid information during training (each position embedding is simply indexed 0, 1, 2, ... in sequence order, per §3.1's flattening step, with no inherent notion of row/column structure built in)?

The empirical answer, found by visualizing learned position embeddings' pairwise similarity after training, is a qualified yes: nearby patches in the original 2D image (even though the model was never told which patches were spatially adjacent) tend to develop noticeably more similar position embeddings than distant patches — the network has discovered, purely from the training signal, an implicit notion of 2D spatial adjacency that was never explicitly provided in the position-indexing scheme itself.

This is a genuinely striking result worth connecting directly back to §6.1's inductive-bias argument: a CNN gets 2D spatial structure built into its architecture for free; a ViT, per this Research Spotlight, can apparently DISCOVER a working approximation of that same structure purely from data — direct empirical evidence for exactly the data-hunger mechanism §6.2 develops, since discovering this structure evidently requires enough training examples to reveal the pattern reliably.

4.3 The [CLS] Token Convention

Borrowed directly from BERT-style NLP transformers, a single learnable "classification token" ([CLS]) is prepended to the sequence of patch embeddings before the sequence enters the transformer encoder. This token carries no patch content of its own — it starts as a learned parameter vector, identical for every input

image — but participates fully in every layer's self-attention computation, meaning it can attend to (and be attended to by) every patch token throughout the network's full depth. After the final encoder layer, the [CLS] token's own updated representation is used as the summary representation of the entire image, fed into the final classification head (Section 5.2) — a design choice worth understanding precisely: rather than pooling or averaging every patch token's final representation (a GAP-style approach, echoing Day 29 §4.3's global average pooling), ViT lets the network itself learn, through self-attention, how to aggregate the full image's information into this one dedicated token.

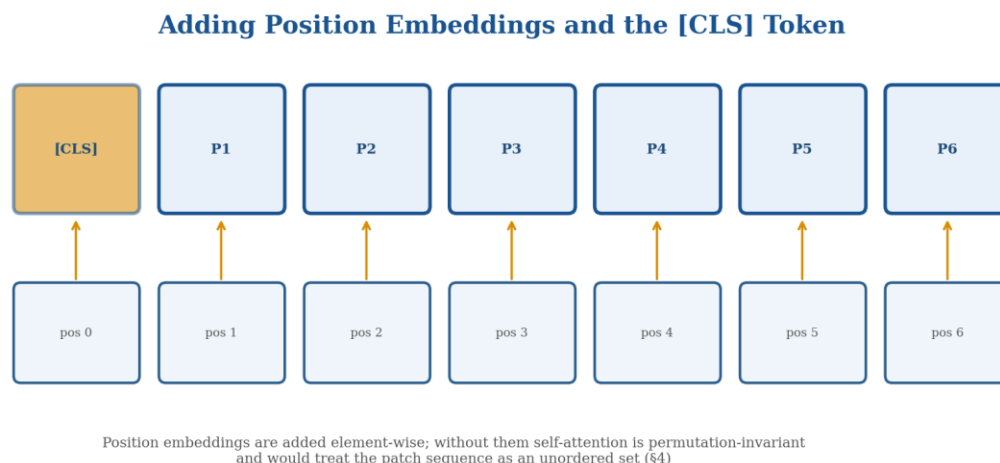


Figure 4. Position embeddings are added element-wise to each patch embedding, restoring the spatial ordering information self-attention's permutation-invariance would otherwise discard entirely (§4.1). The [CLS] token (§4.3), prepended to the sequence before this addition, receives its own learned position embedding like every other sequence element.

It is worth being explicit about an alternative ViT could have used instead, and did not: averaging every patch token's final-layer representation (global average pooling over the sequence dimension, directly analogous to Day 29 §4.3's spatial global average pooling) is a perfectly viable, simpler alternative to the [CLS] token, and some later ViT variants do exactly this. The [CLS] token's advantage is flexibility: because it is a learned, dedicated token rather than a fixed averaging rule, the network can, in principle, learn a more sophisticated, non-uniform aggregation strategy — attending more heavily to whichever patches turn out to be most relevant for a given image, rather than weighting every patch equally by construction the way a plain average would. Whether this flexibility produces a measurable accuracy advantage in practice is a comparison worth checking empirically for a specific architecture and dataset rather than assuming as a universal truth — both design choices remain in active use across different ViT implementations.



Research Spotlight: [CLS] Token vs. Global Average Pooling in Practice

Direct empirical comparisons between the [CLS] token approach and the global-average-pooling alternative, holding every other architectural choice fixed, have generally found the two perform comparably well on standard benchmarks — neither a decisive, consistent winner, somewhat contrary to what the [CLS] token's greater representational flexibility (§4.3) might initially suggest.

One documented, genuinely interesting difference: global average pooling sometimes trains slightly more stably in from-scratch settings specifically (echoing PP16's Task A directly), since it imposes a simpler, more constrained aggregation rule the network does not need to discover through training the way the [CLS] token's fully-learned aggregation does — a small, additional instance of §6's broader inductive-bias argument recurring at the level of a single architectural sub-choice rather than the whole architecture family.

This is worth knowing directly for PP16's Task A specifically: if a from-scratch implementation trains unusually unstably with the standard [CLS] token design, trying a global-average-pooling classification head as a diagnostic comparison is a reasonable, well-motivated experiment — not a deviation from this day's core architecture, but a documented, legitimate design variant within it.

4.4 Toward 2D-Aware Position Schemes

Both position embedding schemes developed in Section 4.2 treat the patch sequence as a flat, 1D-ordered list — position 0, position 1, position 2, and so on — despite the underlying patches actually forming a 2D grid. A natural refinement, worth flagging here ahead of its fuller development in later architectures, uses explicitly 2D-aware position embeddings — encoding a patch's row and column separately, rather than a single flattened sequence index — giving the network a more direct, less indirectly-learned signal about genuine 2D spatial relationships than Section 4.2's Research Spotlight showed a flat 1D scheme can discover only implicitly through training. Relative position schemes go further still, encoding not a patch's absolute position but its position relative to whichever other patch it is currently being compared against in a given attention computation — a refinement Day 39's hierarchical architectures make more extensive use of, since their windowed attention structure (Day 39 §3) makes relative position within a local window a particularly natural and useful quantity to encode directly.

4.5 Relative Position Encoding: A Closer Look

It is worth developing the relative-position idea introduced at the close of Section 4.4 in slightly more mechanical detail, since it recurs directly in Day 39's architecture. Rather than adding a fixed position vector to each token before the encoder begins (Section 4.2's approach, sometimes called absolute position encoding), a relative position scheme instead modifies the attention computation itself (Section 2.2): when computing the attention score between query i and key j , an additional term — a function of the relative distance $(i - j)$, not either token's absolute position — is incorporated directly into the score. This has a genuine practical advantage worth naming: a model trained with relative position encoding can, in principle, generalize more gracefully to sequence lengths it was not explicitly trained on, since "two tokens three positions apart" is a meaningful, consistent relationship regardless of where in a (possibly longer or shorter)

sequence that pair happens to sit — a property absolute position embeddings do not share automatically, which is part of why Section 8.3's interpolation step is necessary at all for ViT's original absolute-position formulation. The tradeoff is implementation complexity: relative position encoding requires modifying the attention computation itself rather than simply adding a precomputed vector to the input, a genuine engineering cost weighed against the generalization benefit.

5 The ViT Encoder & Classification Head

Sections 2 through 4 developed every individual component; this section assembles them into the complete, working Vision Transformer architecture, end to end, and closes with a fully traced numeric forward pass.

5.1 Stacking Transformer Encoder Blocks

A Vision Transformer stacks L identical transformer encoder blocks (Section 2.5, Figure 2), applied sequentially to the patch-and-position-embedded sequence with its prepended [CLS] token (Sections 3 and 4). Each block's residual connections (Section 2.5) mean the sequence's overall shape — length and embedding dimension — remains constant throughout the entire stack; only the content of each token's representation is progressively refined, layer by layer, as self-attention repeatedly re-computes each token's representation as a function of every other token's current representation. This constancy of shape is worth stating explicitly since it differs from Day 29 §6.4's CNN pattern, where spatial dimensions shrink and channel depth grows with network depth — a ViT's sequence length and embedding dimension are instead both fixed at the network's input and preserved throughout its full depth, with all of the network's representational refinement happening entirely within that fixed-size representation rather than through any progressive spatial compression.

5.2 The Classification Head

After the final encoder block, the [CLS] token's representation (Section 4.3) — and only the [CLS] token's representation; every patch token's final representation is simply discarded at this stage — is passed through a small multi-layer perceptron (commonly a single linear layer for fine-tuning, or a slightly larger MLP with one hidden layer for training from scratch) that projects it down to the number of target classes, producing the final classification logits exactly as Day 29 §6.1's classic CNN template's final fully-connected layer did, just operating on a transformer's [CLS] token output rather than a CNN's flattened final feature map.

The Full ViT Architecture, End to End

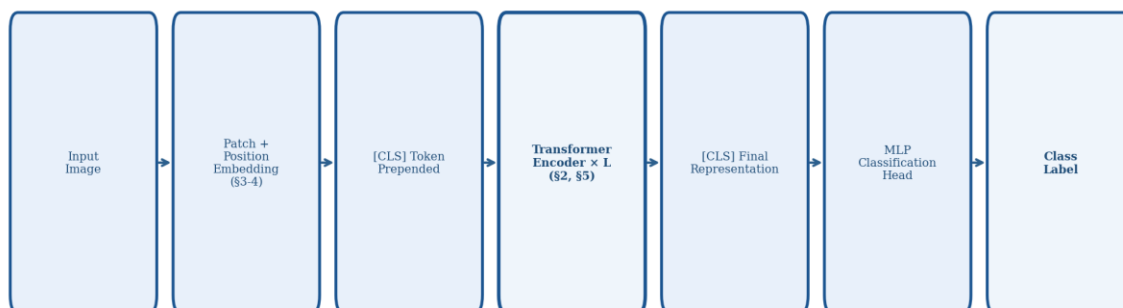


Figure 5. The complete ViT architecture: an input image is patch-and-position-embedded with a prepended [CLS] token, processed through L stacked transformer encoder blocks, and the [CLS] token's final representation is passed through a small MLP classification head to produce the predicted class label.

5.3 Parameter Count: ViT vs. a Comparably-Sized CNN

It is worth grounding this architecture in a concrete parameter-count comparison, echoing Day 30's own repeated practice of loading real architectures and inspecting their actual parameter counts directly rather than reasoning about scale in the abstract. ViT-Base has approximately 86 million parameters — closely comparable to ResNet-152's approximately 60 million parameters (Day 30 §4.3) or a mid-sized EfficientNet variant (Day 30 §6.4), not an order of magnitude larger or smaller. This is a deliberately important point: ViT's eventual data-hunger disadvantage, developed fully in Section 6, is not explained by ViT simply having more parameters to fit — a comparably-sized CNN, per this parameter-count comparison, does not show a comparable data-hunger problem. The explanation lies elsewhere: specifically, in the presence or absence of a built-in inductive bias, which Section 6 now develops as this day's conceptual core.

Architecture	Approx. Parameters	Core Operation	Built-In Spatial Bias?
ResNet-152 (Day 30 §4.3)	~60 million	Convolution	Yes — locality, weight sharing
EfficientNet-B5 (Day 30 §6.4)	~30 million	Depthwise separable convolution	Yes — locality, weight sharing
ViT-Base	~86 million	Self-attention	No — learned entirely from data

5.4 The Named Configuration Family

Exactly as Day 30 §4.3 named the ResNet family (ResNet-18 through ResNet-152) by depth, ViT's own configuration family is named by a combination of embedding width and encoder depth. "ViT-Small," "ViT-Base" ($L=12$ blocks), "ViT-Large" ($L=24$), and "ViT-Huge" ($L=32$) are the standard named

configurations. Figure 13 lays out the family's parameter scaling directly, and it is worth reading the right panel specifically: depth and width both increase together across the family, not independently — a design pattern worth remembering when Section 10.3 develops scaling-law research for exactly this configuration family.

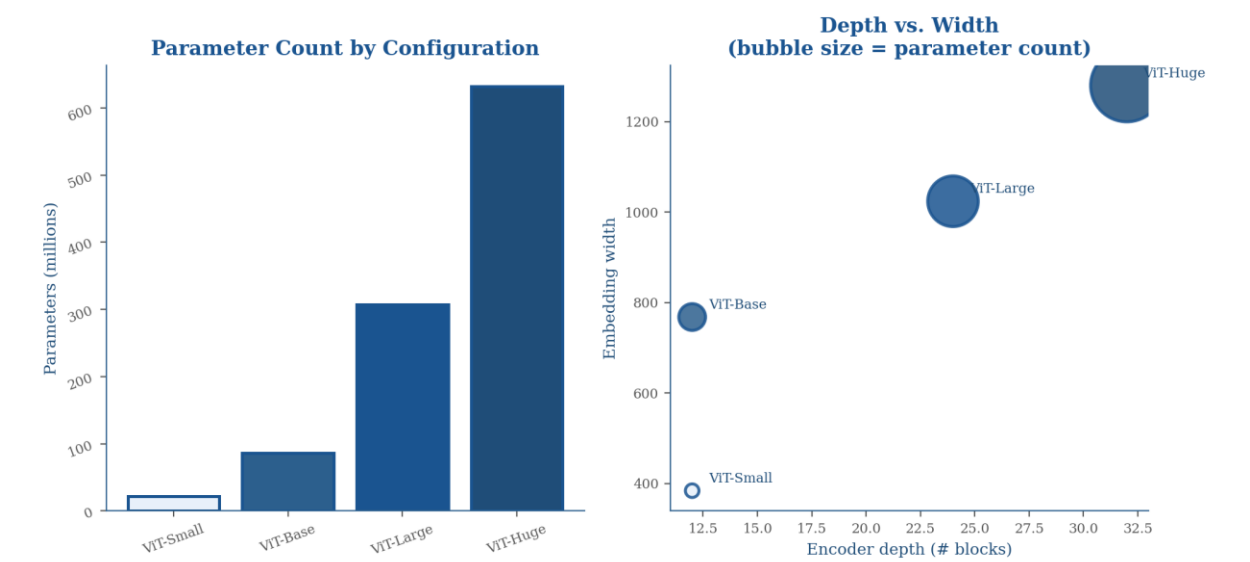


Figure 13. The ViT configuration family, scaling from Small through Huge. Left: parameter count grows super-linearly as both depth and width increase together. Right: the same four configurations plotted by depth versus width, with bubble size showing total parameter count — depth and width increase in tandem across the family, not independently.

Configuration	Encoder Depth (L)	Embedding Width (d)	Approx. Parameters
ViT-Small	12	384	~22 million
ViT-Base	12	768	~86 million
ViT-Large	24	1024	~307 million
ViT-Huge	32	1280	~632 million

Choosing among these configurations follows a familiar pattern from Day 30's own architecture surveys: ViT-Small suits tight compute budgets and smaller downstream datasets, where the largest configurations' extra capacity offers little benefit and considerable extra cost; ViT-Base is the field's de facto default, appearing as the standard choice in the majority of published fine-tuning workflows and pretrained-checkpoint releases (§8.1); ViT-Large and ViT-Huge are reserved for settings with both the compute budget and the downstream data scale (§6.3's crossover consideration applies to configuration size, not only to the CNN-versus-ViT choice itself) to actually benefit from the additional capacity — a smaller dataset fine-tuned on ViT-Huge risks the same overfitting Day 29 §8.1 warned against generally, now specifically because the larger configuration's greater capacity has more room to memorize a small fine-tuning set rather than generalize from it.

5.5 A Complete Forward Pass, Traced End to End

It is worth tracing a complete forward pass through the entire architecture numerically, tying together every mechanism developed in Sections 2 through 5 into one continuous worked example, using ViT-Base's standard configuration and Section 3.4.1's 224×224 -at- 16×16 -patches setup throughout.

Worked Example: A Complete Forward Pass

Step 1 (§3.1–3.2): a $224 \times 224 \times 3$ input image is divided into 196 non-overlapping 16×16 patches, each flattened to a 768-dimensional vector and linearly projected to the working embedding dimension (768 for ViT-Base) — producing a 196×768 matrix.

Step 2 (§4.3): a learned [CLS] token (a single 768-dimensional vector) is prepended, extending the sequence to 197×768 .

Step 3 (§4.1–4.2): a learned position embedding, one 768-dimensional vector per position, is added element-wise to all 197 tokens — the sequence shape remains 197×768 throughout, per §5.1's shape-preservation property.

Step 4 (§2.2–2.5, §5.1): this 197×768 sequence passes through 12 stacked transformer encoder blocks (ViT-Base's $L=12$) — each block computes multi-head self-attention (§2.3) across all 197 tokens, applies a feed-forward network, and preserves the 197×768 shape via its residual connections throughout.

Step 5 (§5.2): after the final (12th) block, only the [CLS] token's row — a single 768-dimensional vector, now having attended to and been informed by every one of the 196 patch tokens across 12 full layers of self-attention — is extracted and passed through the classification head, a small MLP projecting down to the target number of classes.

Total parameter count across every step: approximately 86 million, matching §5.3's ViT-Base figure — the patch embedding projection, 12 encoder blocks' worth of Q/K/V/feed-forward weights, and the final classification head, all accounted for.

5.6 Memory and Compute Footprint at Inference

It is worth developing one further practical dimension beyond Section 5.3's parameter count: the memory required to actually run a forward pass, which depends not only on parameter count but on the size of every intermediate activation the network computes along the way — a distinction that matters because two architectures with identical parameter counts can have quite different memory footprints during actual use. Self-attention's own intermediate computation (Section 2.2) requires storing an $N \times N$ attention score matrix for every head at every layer — for ViT-Base's 197-token sequence, this is a 197×197 matrix per head, a comparatively modest size at this particular sequence length, but one that grows quadratically exactly as Section 2.7 and Section 3.4.1 both established for attention's compute cost, meaning this specific memory cost becomes considerably more significant at the longer sequence lengths a smaller patch size or higher input resolution would produce. A CNN's own intermediate activations, by contrast, scale with spatial resolution and channel depth at each layer (Day 29 §6.4's pattern of shrinking spatial size and growing channel depth), following a quite different memory-scaling profile that does not share this specific

quadratic-in-sequence-length term at all — a genuine, architecture-specific consideration worth checking directly (via a framework's own memory-profiling tools) when deploying either architecture family under a tight memory budget, rather than assuming parameter count alone predicts memory usage.

6 ViT vs. CNN: Inductive Bias Revisited

Day 30 §7.2 introduced this section's core argument briefly, in the course of a broader architecture survey. This section is that brief mention's full, dedicated treatment — the conceptual core of this entire day, and the reason PP16's Task A versus Task B comparison, developed in full in this day's assignment brief, is structured the way it is.

6.1 What "Inductive Bias" Means Here, Precisely

An inductive bias is an assumption built into a learning algorithm's structure, before any training data is seen at all, that narrows the space of functions the algorithm can readily learn — narrowing that helps, provided the assumption happens to be correct for the actual problem at hand. A CNN's inductive bias, developed at length across Day 29 §2.4 and Day 29 §3, is twofold: locality (a unit's receptive field is restricted to a small neighborhood, at least in early layers) and translation equivariance (the same learned pattern is detected identically regardless of where in the image it appears, a direct consequence of weight sharing). Both assumptions are, empirically, very often correct for natural images — nearby pixels usually are more related than distant ones, and a useful visual pattern usually is just as useful wherever in the image it happens to appear — which is exactly why this bias helps rather than hurts.

It is worth situating this specific pair of biases within the more general concept, since 'inductive bias' recurs across machine learning well beyond this day's specific comparison. Every learning algorithm has some inductive bias, whether explicit or implicit — a linear regression model's bias is that the target is well-approximated by a weighted sum of the input features; a decision tree's bias is that the target can be well-approximated by a sequence of axis-aligned splits. The question is never whether a bias exists, but whether the specific bias built into a given architecture happens to match the actual structure of the problem being solved — a CNN's locality-and-equivariance bias matches natural images very well, which is a large part of why convolution dominated computer vision for as long as it did (Days 29 and 30's entire arc), and why removing that bias entirely (this day's ViT) was, at the time, a genuinely uncertain architectural bet rather than an obviously correct next step, exactly as Section 1.3 already flagged.

A Vision Transformer, per Sections 2 through 5, has neither assumption built into its architecture at all. Self-attention (Section 2.2) can, from the very first layer, relate any two patches regardless of their spatial distance, with no architectural preference for nearby patches over distant ones — the entire spatial relevance structure, including whether locality matters at all, must be discovered from data during training, via the position embeddings (Section 4.2) and the attention weights themselves. This is Figure 1's core visual

distinction made mathematically precise: a CNN's kernel structurally cannot do what Figure 1's right panel shows; a ViT's self-attention structurally has no restriction against it.

6.2 Why This Makes ViT More Data-Hungry

A useful way to think about this tradeoff directly: a CNN's built-in locality and translation-equivariance assumptions function as a kind of free prior knowledge, supplied by the architecture itself rather than learned from examples — the network does not need to discover, from scratch, across many training examples, that nearby pixels tend to be related; that fact is already baked into the convolution operation before a single training example is seen. A ViT has no such free prior — it must instead discover, purely from the statistical patterns present in its training data, that (for most natural images, most of the time) something functionally similar to locality is a useful assumption to approximate. Discovering a useful pattern from data requires enough data to reveal that pattern reliably — and this is the precise mechanism behind ViT's empirically observed data-hunger: not a vague "transformers need more data" folk claim, but a specific, mechanistically grounded consequence of the architecture lacking a bias the CNN family gets for free. Section 4.2's Research Spotlight offered direct empirical support for this exact mechanism: learned position embeddings do eventually discover an implicit approximation of 2D spatial adjacency, but only after seeing enough training examples for that pattern to become statistically apparent.

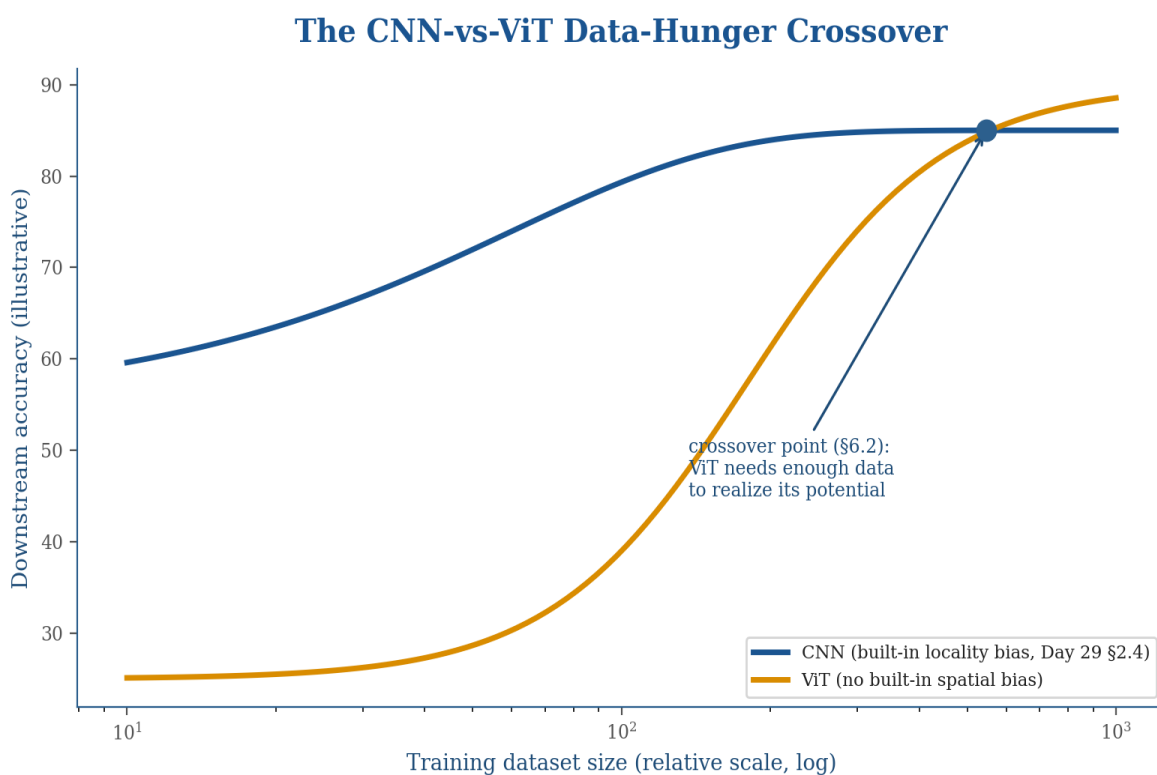


Figure 6. The empirical crossover this section's argument predicts: at small training-dataset scale, a CNN's built-in locality bias gives it a substantial advantage — it does not need to discover from data what the architecture already

assumes. At large enough scale, a ViT has seen enough examples to learn a comparably useful (or better) spatial relevance structure on its own, and its greater architectural flexibility can then match or exceed a CNN's performance.

6.3 The Practical Crossover Point

ViT underperforms comparably-sized CNNs when trained from scratch on small-to-moderate datasets (Section 7 develops exactly how the original ViT paper first demonstrated this, and what data scale was needed to overcome it), and matches or exceeds CNN performance once enough training data is available. "Enough" is not a fixed, universal number — it depends on the specific ViT and CNN variants being compared, the visual diversity of the training data, and the downstream task — but the qualitative crossover pattern Figure 6 illustrates is remarkably consistent across the controlled comparisons this section's Research Spotlight develops below.

Research Spotlight: Controlled Data-Scale Comparisons

The most convincing empirical evidence for Section 6.2's argument comes from controlled experiments that hold architecture size roughly fixed while varying only training dataset scale — isolating the data-hunger effect from confounding differences in model capacity.

Such studies consistently find that a ViT trained from scratch on a small dataset (on the order of ImageNet's ~1.2 million images, per Day 26 §6.3, or smaller) underperforms a comparably-sized CNN trained on the identical data — but that this gap narrows and eventually reverses as training data scale grows into the hundreds of millions of images, the scale the original ViT paper's JFT-300M pretraining dataset (§7.1) was specifically chosen to reach.

This is precisely the empirical foundation PP16's Task A vs. Task B comparison asks you to reproduce directly, at a necessarily much smaller scale: Task A's from-scratch ViT, trained on a compact dataset, is expected to underperform Task B's fine-tuned, already-pretrained ViT — not because Task A is implemented incorrectly, but because it is deliberately demonstrating this section's data-hunger argument on your own results.

6.4 Beyond Locality: Compositionality and Equivariance

Section 6.1's locality-and-translation-equivariance framing, while the most commonly cited CNN inductive bias, is not the complete picture, and it is worth developing two further, related biases explicitly. Hierarchical compositionality is an implicit bias a stacked CNN architecture encodes structurally: early layers detect simple, local patterns (edges, per Day 29 §9.1); middle layers combine these into intermediate patterns (textures, simple shapes); late layers combine those into complex, object-level patterns — a hierarchy directly built into the network's layered structure, since each layer's receptive field is, by construction, built from the previous layer's outputs. A ViT's self-attention, computing global relationships from the very first layer (per Section 6.1), has no comparably explicit hierarchical bias built in — Section 4.2's Research Spotlight showed a flat position-indexing scheme can eventually discover something resembling spatial structure, but a genuinely hierarchical, part-whole compositional structure is a further,

separate pattern the network must also discover purely from data, with no architectural scaffolding pushing it toward that specific hierarchical organization the way stacked convolutional layers do.

Translation equivariance (Day 29 §2.4) — not merely invariance — is worth distinguishing precisely, since the terms are sometimes used loosely. Equivariance means shifting the input shifts the output correspondingly (a feature detected at position (x, y) in the input produces a response at the corresponding shifted position in the output feature map); invariance means the final output is unaffected by the shift entirely (the same class prediction, regardless of where in the image the relevant object appears). A CNN's convolutional layers are equivariant by construction; the network's overall invariance to translation (a cat is still classified as a cat wherever it appears in the frame) emerges from equivariant convolution combined with pooling operations (Day 29 §4) that discard exact position information. A ViT has neither property built in structurally — both approximate translation equivariance and eventual translation invariance, if the trained network exhibits either at all, must again be discovered from data rather than supplied by the architecture.

6.5 The Cost Side of the Ledger

It would be a mistake to read Sections 6.1 through 6.4 as an unqualified case against ViT — every inductive bias is a double-edged assumption, and it is worth stating the CNN side's own cost explicitly, not only its benefit. A CNN's locality bias, precisely because it is a genuine structural assumption baked into the architecture rather than a preference the network could override if the data suggested otherwise, imposes a hard ceiling on what the network can represent efficiently: a pattern that genuinely requires relating two very distant image regions directly (not merely through many intermediate layers' worth of gradually-expanding receptive field, per Day 29 §3.3) is awkward for a CNN to represent at all, no matter how much training data is available, since the architecture itself restricts early-layer computation to local neighborhoods. A ViT's lack of this restriction is precisely Section 6.1's costly-at-small-scale, valuable-at-large-scale flexibility: once enough data is available to compensate for the missing free prior, that same missing restriction becomes an advantage rather than a liability, since the network is free to discover and represent genuinely long-range relationships a CNN's architecture would make comparatively difficult to represent at any data scale. This is the precise sense in which Section 6.3's crossover point is not merely "ViT eventually catches up" but "ViT eventually gains a structural advantage a CNN's own architecture cannot offer regardless of data scale" — worth holding in mind as the complete, two-sided version of this section's central argument.



Worked Example: A Toy Sample-Efficiency Calculation

It is worth making §6.2's data-hunger argument concrete with an illustrative, deliberately simplified numeric picture — not a claim about exact real-world figures, but a way of building intuition for why the crossover in Figure 6 has the shape it does.

Suppose a CNN's built-in locality bias means it needs to see roughly N examples of a given visual pattern before reliably learning to exploit it, since much of the 'nearby pixels are related' structure is already supplied by the architecture and the network mainly needs to learn WHICH local patterns matter for the task.

A ViT, needing to discover both which patterns matter AND that locality itself is a useful assumption to approximate (per §6.2), might need something closer to $k \times N$ examples for the same reliability, where k captures the 'extra discovery burden' of having no free architectural prior — illustratively, k might be small (close to 1) for a task where long-range relationships genuinely matter and locality is a poor assumption anyway, since the ViT is not fighting against a bias that turns out to be wrong for this specific task, but might be considerably larger for a task where locality genuinely is the dominant useful pattern, since the ViT must expend some of its training signal discovering a structural fact the CNN received for free.

This toy k factor is exactly what Figure 6's crossover point represents empirically — not a fixed universal constant, but a task- and architecture-pair-specific quantity that Section 6.3's controlled comparisons measure directly rather than derive analytically, and that PP16's own Task A vs. Task B comparison gives you a direct, if small-scale, empirical window into for your own chosen dataset.

6.6 A Connection to Few-Shot and Meta-Learning

Section 6.2's data-hunger argument connects to a broader question worth naming explicitly: how does a learning algorithm's inductive bias interact with the few-shot setting, where only a handful of labeled examples are available for a new task? A CNN's locality bias, per Section 6.1, provides useful structure even with very few examples — the architecture already assumes something correct about the problem, so few examples are needed to fine-tune around that assumption. A ViT trained from scratch would be a particularly poor few-shot learner for exactly Section 6.2's reasons — with almost no data to discover useful spatial structure from at all, a from-scratch ViT has essentially no useful prior to fall back on. This is precisely why Section 7.5's transfer-learning default matters so much for ViT specifically: a pretrained ViT checkpoint effectively converts a few-shot problem into a fine-tuning problem, since the large-scale pretraining (Section 7.1) has already done the expensive work of discovering useful visual structure, leaving only a comparatively small adaptation to the new task's specific categories — the few-shot setting is, in a real sense, the most extreme possible illustration of why Section 7.5 identifies transfer learning as the practical default rather than an optional convenience.

7 Training ViT: Data Requirements & Tricks

Section 6 developed why ViT is more data-hungry than a comparably-sized CNN; this section develops both the original solution (simply using enough data) and the practical tricks that reduce this requirement for settings where JFT-300M-scale data is not available — which, realistically, describes nearly every

setting outside a small number of large industrial labs — plus the optimizer and precision considerations that make ViT training numerically well-behaved in the first place.

7.1 The Original Approach: Massive-Scale Pretraining

ViT's original formulation (Dosovitskiy et al., 2020) addressed Section 6's data-hunger problem the most direct way possible: pretrain on JFT-300M, a dataset of roughly 300 million labeled images — two orders of magnitude larger than ImageNet's 1.2 million (Day 26 §6.3) — and only then fine-tune on smaller downstream datasets. At this scale, Section 6.3's crossover point is comfortably exceeded, and the resulting pretrained ViT matched or exceeded the best contemporary CNNs on standard benchmarks. This approach's obvious practical limitation is that JFT-300M-scale data and the training compute needed to pretrain on it are simply unavailable outside a small number of large industrial research labs — motivating every technique the remainder of this section develops.

7.2 Strong Data Augmentation

Aggressive data augmentation — a direct, intensified application of the augmentation techniques Day 27 §6 developed for CNN training — is one practical lever for narrowing the data-hunger gap without requiring JFT-300M-scale data. The reasoning connects directly back to Day 27 §6.1's original augmentation rationale: augmentation artificially expands a finite dataset's effective diversity, and a ViT specifically benefits from this expanded diversity because, per Section 6.2's argument, it must learn spatial relevance patterns from the sheer volume and variety of examples it sees — augmentation increases both without requiring any genuinely new data collection at all. In practice, ViT training recipes typically apply RandAugment or a similarly comprehensive augmentation policy (Day 27 §6.3's RandAugment/AutoAugment discussion applies directly here) considerably more aggressively than a comparable CNN training recipe would, precisely because a ViT has more to gain from the additional effective data diversity than a CNN, whose built-in inductive bias already supplies part of what augmentation would otherwise need to teach it.

Real-World Use Case: Augmentation Strength as a Direct Data-Hunger Lever

A team fine-tuning a ViT on a genuinely small, specialized dataset (a few thousand images of a narrow product category, say) found that a standard, comparatively mild CNN-style augmentation policy — small random crops and horizontal flips, per Day 27 §6.2's classic augmentations — left validation accuracy noticeably behind a comparable CNN baseline trained on the identical data.

Switching to a considerably more aggressive policy — RandAugment with a higher magnitude setting, plus Mixup and CutMix (Day 27 §6.3) layered on top — closed much of this remaining gap, consistent with §6.2's argument: the ViT specifically needed the additional effective data diversity aggressive augmentation provides, in a way the CNN, with its built-in locality prior already partially compensating for limited data, needed considerably less.

This is a genuinely useful diagnostic pattern worth remembering directly: if a from-scratch or lightly-pretrained ViT underperforms a CNN baseline by more than §6.3's typical crossover pattern would predict at your dataset's scale, checking augmentation strength specifically — before assuming a deeper architectural or optimization problem — is often the more productive first step.

7.3 Regularization

Stronger regularization than a comparable CNN training run would typically use — higher dropout rates (Day 29 §8.2), increased weight decay (Day 29 §8.4) — helps compensate for ViT's lack of a built-in inductive bias by more aggressively discouraging the kind of training-data memorization Day 29 §8.1 identified as overfitting's defining symptom, since a network with no free architectural prior has, in a real sense, more genuinely arbitrary flexibility available to overfit with in the first place. Stochastic depth — randomly dropping entire transformer encoder blocks during training, with all blocks used at inference — is a further regularization technique particularly well-suited to a stacked-block architecture like ViT's, forcing the network to avoid over-relying on any single block's specific contribution, the same underlying "discourage over-reliance on any one component" logic Day 29 §8.2's dropout applied at the level of individual units rather than entire blocks.

7.4 Knowledge Distillation From a CNN Teacher: DeiT

DeiT (Data-efficient image Transformers, Touvron et al., 2021), previewed briefly here ahead of its full development in Day 39 §5, takes a structurally different approach: rather than relying purely on data scale or augmentation, a pretrained CNN "teacher" model's predictions are used as an additional training signal for the ViT "student" — the student is trained not only to predict the correct label, but to match the teacher CNN's own output distribution, via a dedicated distillation token architecturally similar to the [CLS] token (Section 4.3). Because a CNN's built-in inductive bias (Section 6.1) gives it a head start on smaller datasets, distilling from a CNN teacher effectively transfers some of that architectural advantage to the ViT student indirectly, without requiring the ViT itself to discover the same useful spatial patterns purely from raw data volume — DeiT's own published results showed a from-scratch ViT trained with this distillation signal reaching competitive ImageNet accuracy using only ImageNet-1k's ~1.2 million images, without any JFT-300M-scale pretraining at all, a genuinely significant practical result for exactly the compute-constrained setting most readers of this compendium will actually be working in.

7.5 Transfer Learning: The Practical Default

For the overwhelming majority of practical applications, none of the above is actually necessary to implement directly — transfer learning, the same workflow Day 29 §7.5 developed as the practical default for CNNs, applies identically to Vision Transformers: start from a checkpoint someone else has already pretrained at large scale (echoing Section 7.1's JFT-300M pretraining, but done once by the checkpoint's

original creators, not by every downstream user), and fine-tune on your own, typically much smaller, target dataset. This is precisely PP16's Task B, and precisely why Task B is expected to substantially outperform Task A's from-scratch training within any single Colab session's practical time budget — not a flaw in Task A's implementation, but Section 6.2's data-hunger argument playing out exactly as predicted.

Technique	Addresses	Relative Effort	Typical Setting
Massive-scale pretraining (§7.1)	Data-hunger directly, at the source	Very high (industrial-scale compute)	Original checkpoint creation only
Strong augmentation (§7.2)	Effective data diversity	Low — a training-recipe change	Any from-scratch training run
Extra regularization (§7.3)	Overfitting on limited data	Low — a hyperparameter change	Any from-scratch training run
Distillation / DeiT (§7.4)	Data-hunger via a CNN teacher's bias	Moderate — needs a trained teacher	From-scratch training with a teacher available
Transfer learning (§7.5)	Data-hunger by avoiding it entirely	Low — reuses existing work	The practical default for most applications

7.6 Optimizer Choices and Learning Rate Schedules

It is worth developing ViT's specific optimizer sensitivity in more depth than a passing mention allows, since it is a genuinely common source of training difficulty for a first implementation. AdamW — the standard Adam optimizer (already used throughout this compendium since Day 29 §7.3) combined with decoupled weight decay — is the near-universal default for transformer training generally, including ViT, in preference to the plain SGD-with-momentum that remains a perfectly reasonable choice for CNN training. Transformers, empirically, are considerably more sensitive to learning rate than a typical CNN, and a learning rate warmup phase — starting training at a very small learning rate and increasing it linearly over the first several hundred to few thousand steps, before switching to a decay schedule (commonly cosine decay) for the remainder of training — has become a standard, close-to-mandatory component of any ViT training recipe, from-scratch or fine-tuning alike. Figure 15 shows a representative schedule.

A Typical ViT Learning Rate Schedule

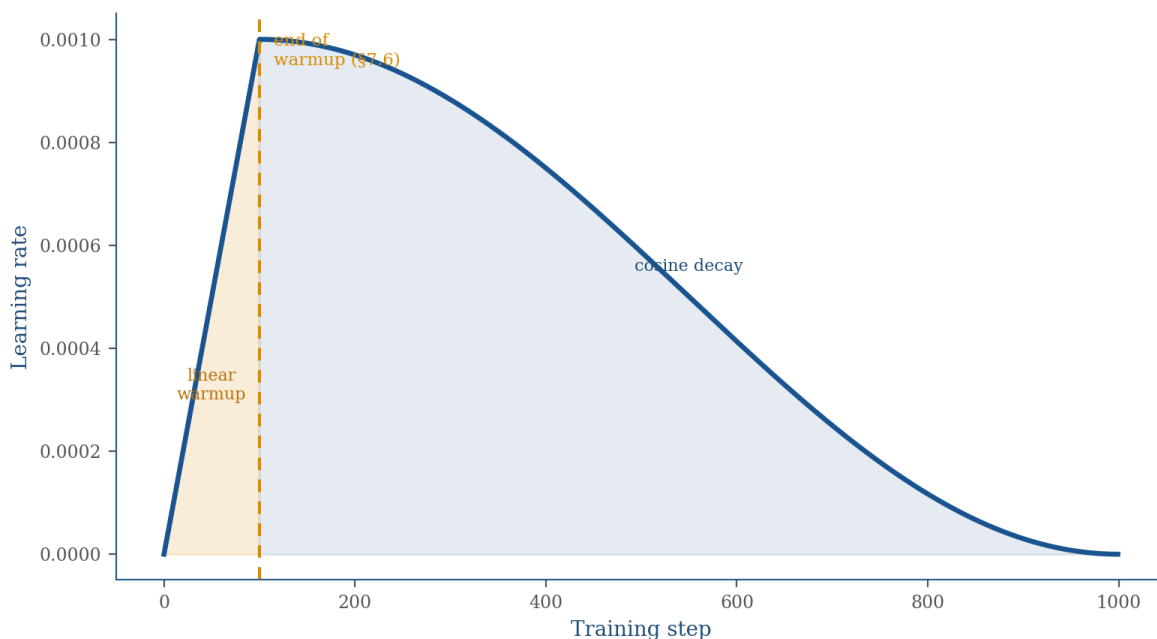


Figure 15. A typical ViT learning rate schedule: a linear warmup phase over the first ~10% of training, followed by cosine decay for the remainder. Omitting warmup entirely is a common and easily-diagnosed cause of early training instability specifically for transformer architectures — a difficulty CNN training recipes are typically considerably more forgiving about.

Why transformers specifically need this warmup phase, where a typical CNN training recipe can often proceed without one, is worth understanding mechanistically rather than treating as an arbitrary convention. Early in training, before the network's weights have settled into a reasonable range, self-attention's softmax computation (Section 2.2, Section 2.4) can be particularly sensitive to large gradient updates — an overly large early learning rate can push the attention mechanism into a poorly-conditioned regime it struggles to recover from, a specific numerical fragility that a gradual warmup phase is specifically designed to avoid by keeping early updates small until the network's weights have stabilized somewhat.

7.7 Mixed-Precision Training Considerations

Mixed-precision training — performing the bulk of training computation in a lower-precision numeric format (commonly 16-bit floating point) rather than the standard 32-bit format, while retaining select operations in full precision where numerical stability requires it — is a general deep learning training technique, not ViT-specific, but is worth mentioning directly here because it interacts with Section 2.4's scaling-factor discussion in a genuinely practical way: self-attention's softmax computation, and the large intermediate values that can arise before scaling is applied, are a common source of numerical overflow specifically when training in reduced precision, making careful attention to exactly where full-precision computation is retained (typically the softmax operation itself, even when surrounding operations run in reduced precision) a practically important detail for large-scale ViT training runs, though one that

pretrained-checkpoint fine-tuning (Section 7.5, Section 8) typically shields a downstream user from needing to manage directly, since the original pretraining run already navigated these numerical considerations before the checkpoint was published.

8 ViT in Practice

Sections 2 through 7 developed the architecture and its training considerations from first principles; this section develops the practical mechanics of actually working with a Vision Transformer in a typical project — directly setting up PP16's Task B, and closing with a practical demonstration of the attention-visualization capability this day's architecture provides essentially for free.

8.1 Using Pretrained ViT via Hugging Face and timm

The Hugging Face `transformers` library and the `timm` (PyTorch Image Models) library, previewed already in Day 30 §5.4's Neural Architecture Search discussion as a source of pretrained checkpoints, both provide ready-to-use pretrained ViT checkpoints — `google/vit-base-patch16-224` is a standard, widely-used starting point, pretrained on ImageNet-21k and fine-tuned on ImageNet-1k, directly loadable with a small number of library calls rather than requiring Section 7.1's JFT-300M-scale pretraining to be repeated by every downstream user. The checkpoint's name itself, per Section 3.4.1's closing observation, encodes its configuration directly: "base" (Section 5.4's ViT-Base), "patch16" (16×16 patches, per Section 3.1), and "224" (expected 224×224 input resolution) — worth reading deliberately when selecting a checkpoint, since mismatching any of these three encoded parameters against your actual usage is a common, avoidable source of the shape-mismatch and interpolation errors Section 8.3 and Section 8.4 both develop.

Both libraries follow a broadly similar high-level pattern, worth knowing at a conceptual level even without memorizing exact function signatures: a single call loads a named pretrained checkpoint (returning a ready-to-use model with its original pretrained classification head intact), a companion call retrieves the exact preprocessing pipeline that checkpoint's own pretraining used (directly addressing Section 8.4's normalization-mismatch pitfall by construction, provided this companion call is actually used rather than a hand-assumed preprocessing pipeline), and the model's final classification layer can then be replaced with a freshly-initialized layer sized to the target number of classes before fine-tuning begins (Section 8.2). The `timm` library specifically maintains an unusually broad catalog of checkpoint variants — different configurations (Section 5.4), different patch sizes and resolutions (this section's table above), and checkpoints from several different original training recipes — making it a particularly useful single source to check when a project's specific requirements (a particular resolution, a particular efficiency tradeoff) do not match the single most common `vit-base-patch16-224` default.

8.2 The Fine-Tuning Workflow

Fine-tuning follows the identical structural pattern Day 29 §7.5 established for CNNs: load the pretrained checkpoint, replace the final classification head (Section 5.2) with one matching your target number of classes, and continue training — typically for comparatively few epochs, at a comparatively low learning rate (echoing Section 7.6's warmup discussion, though typically a shorter, gentler warmup than a from-scratch run requires, since the pretrained weights are already in a well-conditioned range), since the pretrained weights already encode broadly useful visual representations that only need modest adjustment for the new task, exactly the same reasoning Day 29 §7.5 and Day 37 §8.1 both applied to their own respective fine-tuning scenarios. A further practical choice worth naming explicitly: fine-tuning can either update every one of the network's parameters ("full fine-tuning") or freeze the pretrained encoder entirely and train only the new classification head ("linear probing") — full fine-tuning typically achieves higher final accuracy given enough target-task data, while linear probing is faster, cheaper, and less prone to catastrophically overwriting the pretrained representation's useful general-purpose structure on a very small target dataset, a genuine tradeoff worth considering explicitly for PP16's Task B rather than defaulting unreflectively to full fine-tuning.

8.3 Position Embedding Interpolation at a Different Resolution

A ViT-specific complication worth understanding directly: Section 4.2 established that ViT's original formulation uses learned position embeddings, with one embedding vector per patch position at the specific resolution the checkpoint was originally pretrained at. Fine-tuning at a different input resolution than pretraining — a common and often beneficial practice, since a higher resolution generally improves downstream accuracy at some additional compute cost — changes the number of patches (per Section 3.4.1's worked example, a different image size produces a different patch grid), meaning the pretrained position embeddings no longer align one-to-one with the new patch count. The standard fix interpolates the original, pretrained position embeddings onto the new, different-sized grid — treating the original learned embeddings as samples of a continuous underlying spatial function that can be resampled at a new resolution, conceptually similar in spirit (though mechanically distinct) to Day 27 §2.1's image resizing interpolation.

8.4 Common Pitfalls in Practice

- **Patch size / image size mismatches.** The input image's height and width must be evenly divisible by the patch size (per Section 3.4.1's worked example); an image size that does not divide evenly produces an ill-defined patch grid, typically caught as a shape-mismatch error but worth checking proactively rather than debugging reactively.
- **Forgetting the [CLS] token.** A from-scratch implementation (PP16 Task A) that omits the [CLS] token (Section 4.3) entirely, or fails to correctly extract specifically its final representation for the classification head (Section 5.2) rather than some other token's, will train but will not learn a coherent whole-image summary representation.

- **Position embedding interpolation errors when fine-tuning at a new resolution.** Per Section 8.3, fine-tuning or evaluating at a resolution different from pretraining requires correctly interpolating the position embeddings — an incorrect or omitted interpolation step silently misaligns spatial information without necessarily raising an error.
- **Preprocessing/normalization mismatches.** Per Day 27 §5.2's original warning, a pretrained ViT checkpoint expects input normalized with the specific statistics its pretraining used — using different or default normalization values silently degrades performance without raising any error, exactly the same silent failure mode Day 27 §5.2 documented for CNN backbones.
- **Omitting the learning rate warmup phase.** Per Section 7.6, transformers are more sensitive to an overly large early learning rate than a typical CNN — a training run that diverges or oscillates in its very first steps should prompt a check of the warmup schedule before any other hyperparameter.
- **Defaulting to full fine-tuning without considering linear probing.** Per Section 8.2, a very small target dataset can suffer more from full fine-tuning's capacity to overwrite pretrained structure than it benefits from full fine-tuning's extra flexibility — worth comparing both options directly rather than assuming one is always correct.

8.5 Visualizing Attention Maps in Practice

A genuinely useful practical capability, worth developing directly here as this section's closing topic and as a direct bridge to Day 37's explainability content: because self-attention (Section 2.2) computes an explicit weight for every patch-pair relationship at every layer, a ViT's own attention weights can be extracted and visualized directly, showing which image regions a given layer's [CLS] token (or any other token) attended to most strongly — precisely the kind of built-in explanation signal Day 37 §5.1 previewed in the context of this compendium's broader explainability content, now developed concretely enough to actually use. Figure 14 illustrates, at an illustrative level, how these attention patterns typically evolve across a network's depth.

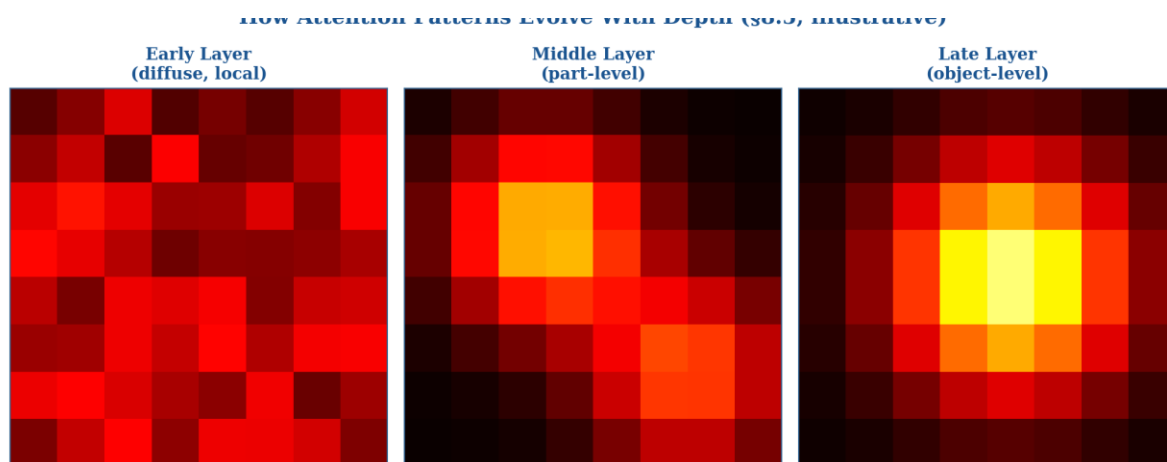


Figure 14. How a ViT's attention patterns typically evolve with depth, illustrated schematically. Early layers tend to show diffuse, relatively local attention — not yet identical to a CNN's hard architectural locality constraint (§6.1), but empirically often resembling it. Middle layers show attention organizing around coherent image parts. Late layers concentrate attention on the specific regions most relevant to the final classification decision — directly recognizable as the same kind of localization signal Day 37 §3.2's Grad-CAM produces through an entirely different, gradient-based mechanism.

It is worth connecting this directly back to Day 37 §5.2's faithfulness caveat rather than presenting attention visualization as a strictly superior alternative to Day 37's gradient-based and perturbation-based methods: high attention weight on a region is suggestive evidence about what the model found relevant, but — exactly as Day 37 §5.2 argued in general — is not guaranteed to be a fully faithful account of the model's actual decision process. In practice, comparing a ViT's raw attention visualization against an independent, gradient-based check (Day 37 §3's Grad-CAM, adapted for a transformer architecture rather than a CNN) is a reasonable way to build additional confidence in a specific explanation before relying on it for a high-stakes judgment — the same cross-checking discipline Day 37 §11's debugging checklist recommended in general, now applied specifically to this day's own architecture.

8.6 Debugging With Small Toy Examples

A practical habit worth establishing explicitly, particularly valuable for PP16's Task A: before training a from-scratch ViT on a real dataset at all, verify every individual component — patch embedding (§3), position embeddings and the [CLS] token (§4), a single encoder block (§2.5) — on a small, synthetic, deliberately toy input where the correct output shape and rough numeric behavior can be checked by hand or with a short assertion, exactly the same component-by-component verification discipline worth applying to any multi-stage pipeline this compendium has built (Day 28 §11's five-stage debugging checklist established this same principle for the classical feature-matching pipeline). A patch embedding step applied to a small, synthetic 32×32 test image should produce a sequence of exactly the expected length and dimension (per §3.4.1's formula) before ever being combined with a full encoder stack; an encoder block's output should preserve the same sequence shape as its input (per §5.1's shape-preservation property) on a synthetic input of any content at all. Catching a shape mismatch or a missing residual connection at this small, cheap, fast-to-debug stage is considerably less costly than discovering the same bug only after a lengthy full training run has already completed and produced a puzzlingly poor result.

8.7 Batch Size and Gradient Accumulation

A final practical consideration worth developing directly, particularly relevant to PP16's runtime guidance (developed in full in this day's assignment brief): Vision Transformers, like most deep networks, generally benefit from a reasonably large batch size during training, both for stable gradient estimates and for making efficient use of GPU parallelism — but Section 5.6's memory footprint discussion means a large batch size directly multiplies the memory required to store every layer's intermediate activations across the entire batch, which can exceed a given GPU's available memory well before a desired batch size is reached, particularly on a T4-class GPU rather than the more generous A100 this day's assignment treats as its target

hardware. Gradient accumulation is the standard practical workaround: rather than computing a gradient update from one large batch, several smaller "micro-batches" are processed sequentially, their gradients summed (accumulated) rather than immediately applied, and only after accumulating gradients from enough micro-batches to match the desired effective batch size is an actual weight update performed — trading additional wall-clock time (several smaller forward/backward passes instead of one large one) for a memory footprint bounded by the smaller micro-batch size rather than the full effective batch size. This is a genuinely useful technique to have available specifically for PP16's graceful T4 fallback (developed in full in Section E of the assignment brief below), letting a training recipe designed around an A100-scale batch size still run, at reduced speed but unreduced effective batch size, on more constrained hardware.

9 Real-World Applications

Vision Transformers are not merely a theoretical alternative to convolution — they have become a standard, widely-deployed component across several of this compendium's earlier application areas, and this section surveys exactly where and why, closing with a broader mosaic view across this entire compendium's application domains.

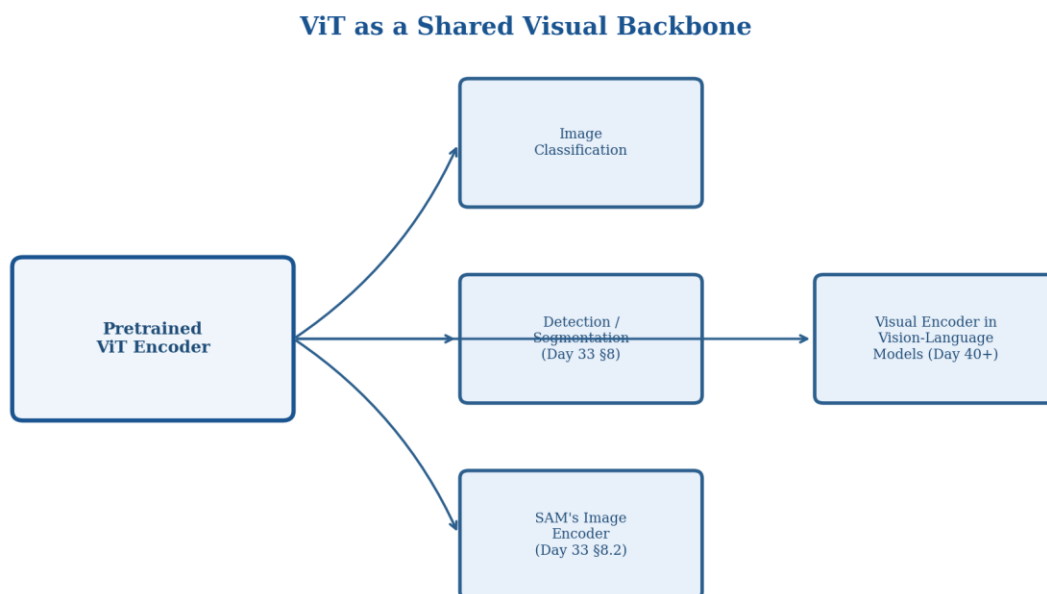


Figure 7. A pretrained ViT increasingly serves as a shared, general-purpose visual backbone across many downstream applications — echoing Day 30 §9's backbone-selection framework, now with a transformer-based option alongside every CNN family that framework originally covered.

9.1 ViT as a Backbone in Detection and Segmentation

Day 30 §6's backbone-selection framework treated backbone choice as a decision to be made per-application, weighing dataset size, latency budget, and fine-tuning strategy — the same framework applies directly here, with a pretrained ViT (or, more commonly in practice, one of Day 39's hierarchical ViT

variants, since a plain ViT's single-resolution output is a poor structural fit for dense prediction) now a genuine option alongside every CNN family Day 30 covered. Day 33 §8.1's SegFormer content previewed exactly this: a transformer encoder paired with a comparatively lightweight decoder for dense segmentation prediction, trading a CNN's gradually-growing local receptive field for a transformer's immediate global one.

It is worth being explicit about exactly why a plain ViT is a poor structural fit for dense prediction, rather than merely asserting it. Object detection (Day 31) and segmentation (Day 33) both benefit substantially from multi-scale feature maps — Day 31 §6.1's SSD architecture, for instance, explicitly predicted from several different feature-map resolutions specifically to handle objects of different sizes well. A plain ViT, per Section 5.1's shape-preservation property, maintains a single, constant sequence length and resolution throughout its entire depth — there is no natural analogue to a CNN's progressively-shrinking spatial feature maps (Day 29 §6.4) from which a detection or segmentation head could draw multi-scale features. This is precisely the structural gap Day 39 §3's hierarchical Swin Transformer closes, by design, and it is worth carrying this specific gap forward as motivation for why Day 39's content is necessary, not merely a further refinement for its own sake.

9.2 Image Classification at Scale

For pure image classification — the task this day's architecture was originally designed around, per Section 5's development — a fine-tuned pretrained ViT (Section 8's workflow) is now a standard, competitive choice alongside the CNN families Day 30 surveyed, with the practical choice between them generally following Day 30 §9's decision framework directly: dataset size and available pretraining (Section 6.3's crossover consideration) as the primary deciding factors, rather than either family being a categorically superior default.

Callout Box: *A concrete illustration of Section 9.2's decision framework in practice: a large-scale image-hosting platform classifying user-uploaded photographs into broad content categories (landscapes, food, portraits, documents) has access to an enormous, continuously-growing volume of training data — precisely Section 6.3's large-data regime where a ViT's crossover advantage applies. A specialized industrial inspection system classifying manufacturing defects from a few hundred labeled example images per defect type, by contrast, sits squarely in Section 6.2's small-data regime, where a CNN's built-in inductive bias — or a heavily-pretrained ViT via transfer learning, per Section 7.5 — remains the more reliable choice. The same underlying task (image classification) receives a different architectural answer purely as a function of available data scale, exactly as Section 6's entire argument predicts.*

9.3 ViT as the Visual Encoder Inside Vision-Language Models

This is the load-bearing connection into the remainder of this module. A vision-language model — the subject of Day 40 onward — needs some mechanism for converting a raw image into a representation that can be related to, and jointly reasoned about with, natural language text. A pretrained ViT, per Section 8.1, already produces exactly this: a sequence of patch-level representations (or, via the [CLS] token, a single whole-image summary representation) suitable for combining with a text encoder's own output. This is

precisely why Day 40 onward assumes fluent command of this day's content — ViT is not merely one option among several for vision-language systems, but the standard, default visual encoder choice for the large majority of the field's current systems.

Callout Box: Day 33 §8.2 introduced SAM (the Segment Anything Model) and its promptable, zero-shot segmentation capability, but deliberately left unexplained exactly how SAM's image understanding actually worked internally. The answer, now available with this day's content in hand: SAM's image encoder is a ViT, pretrained at large scale (per Section 7.1's massive-scale pretraining approach) specifically so that its patch-level representations are rich and general enough to support accurate segmentation for arbitrary, previously unseen prompted regions — a direct, concrete instance of Section 9.1's backbone-choice discussion, and worth revisiting Day 33 §8.2 with this added understanding now available.

9.4 Medical Imaging, Satellite Imagery, and Video

Three further application domains, each drawn directly from this compendium's own earlier days, are worth developing explicitly to show the breadth of ViT's current practical reach. Medical imaging (Day 37) is an active area of ViT adoption, echoing Day 37 §8.2's classification-backbone path directly — a ViT fine-tuned on chest X-rays or similar 2D modalities faces exactly Day 37 §8.2's domain-gap consideration (a natural-image-pretrained checkpoint's early layers transfer more readily than its later, more class-specific layers), and Section 8.5's attention-visualization capability offers a genuinely useful, complementary explainability signal alongside Day 37 §3's Grad-CAM for exactly the high-stakes trust reasons Day 37 §6.2 developed at length. Satellite and aerial imagery (Day 33 §7.1's tiling discussion) presents ViT with a scale challenge related to, but distinct from, Section 8.3's resolution-interpolation concern — a satellite image is frequently far larger than any single ViT's practical input resolution, requiring the same kind of tiling strategy Day 33 §7.1 developed for segmentation, now applied to classification or detection tasks built on a ViT backbone. Video and spatio-temporal ViT variants extend Section 5's architecture across the temporal dimension, echoing Day 36 §8's video-processing content and Day 32 §8.2's 3D CNN approach to activity recognition — a spatio-temporal ViT treats a video clip's patches across both space and time as one extended sequence, letting self-attention relate a patch in one frame directly to a patch in a distant frame, a genuinely different mechanism from Day 32 §8.2's 3D convolutional approach to the identical underlying problem of modeling motion across a video clip.

Real-World Use Case: Spatio-Temporal Attention for Action Recognition

A concrete illustration of Section 9.4's video-ViT discussion worth making specific: consider distinguishing 'picking up a cup' from 'putting down a cup' in a video clip — two actions that can look nearly identical in any single frame, differing only in their temporal ordering, echoing Day 32 §8.1's own

concrete motivating example for why activity recognition needs genuine temporal context at all.

A spatio-temporal ViT's self-attention can directly relate a hand-and-cup patch in an early frame to the corresponding patch in a late frame, regardless of how many frames separate them, without requiring the information to propagate step by step through every intervening frame the way Day 32 §8.2's alternative recurrent or 3D-convolutional approaches would.

This is precisely Section 6.5's cost-side-of-the-ledger argument re-appearing in the temporal domain rather than the spatial one: a video-domain architecture with a built-in temporal locality bias (nearby frames matter most, per a 3D CNN's local temporal kernel) may need to propagate information across many intermediate frames to relate distant ones, exactly the same structural limitation Section 6.5 identified for CNN spatial locality, now recurring across time instead of space.

Domain	Where ViT Fits	Compendium Connection
Detection & segmentation	Backbone, typically hierarchical (Day 39)	Day 33 §8.1's SegFormer
Vision-language models	Standard visual encoder	Day 40 onward, this module
Medical imaging	Classification backbone + attention-based explainability	Day 37 §8.2, §3
Satellite imagery	Backbone, with tiling for large images	Day 33 §7.1
Video / activity recognition	Spatio-temporal self-attention	Day 36 §8, Day 32 §8.2

ViT's Reach Across This Compendium's Application Areas

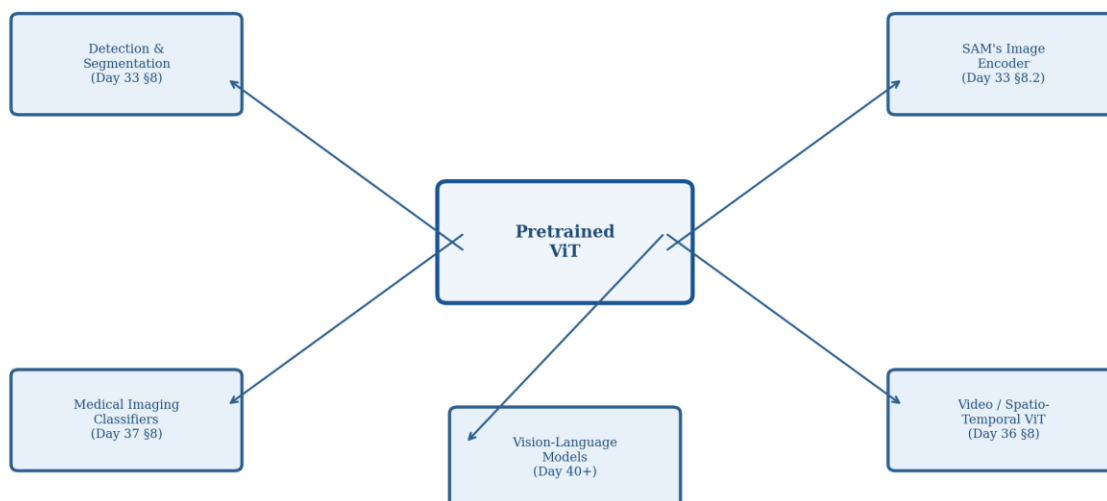


Figure 16. A wider mosaic view of ViT's reach across this compendium's own application areas — detection and segmentation, SAM's promptable segmentation, medical imaging classifiers, video and spatio-temporal modeling, and the vision-language models this module builds toward. A single pretrained encoder increasingly underlies work this compendium once treated as architecturally separate.

9.5 Ethical Considerations: Attention, Trust, and Surveillance at Scale

Every application this section has surveyed carries forward, rather than resets, the ethical considerations this compendium has developed since Day 26 §9 — a new architecture does not create a new ethical starting point, and it is worth being explicit about exactly how this day's specific capabilities interact with those earlier concerns.

Section 8.5's attention-visualization capability is worth revisiting here specifically through Day 26 §9.2's dataset-bias lens: because a ViT's attention weights are directly inspectable, they offer a genuine, practical tool for auditing whether a trained model's decisions are grounded in appropriate visual evidence or in a spurious correlation of exactly the kind Day 29 §9.3's wolves-versus-huskies case and Day 37 §1.1's restated version both warned against — a real advantage worth naming, not merely a caveat-laden capability. At the same time, a pretrained ViT's role as a general-purpose visual encoder (Section 9.3) means any bias baked into its large-scale pretraining data propagates directly into every downstream application built on top of it, echoing Day 26 §9.2's Gender Shades concern at a larger, harder-to-audit scale: a single pretraining run's data choices can affect medical classifiers (§9.4), detection systems (§9.1), and vision-language models (§9.3, Day 40 onward) simultaneously, rather than each downstream application's bias being independently traceable to its own, separately-collected training data the way Day 26 §9.2's original framing implicitly assumed.

Day 31 §9.4's surveillance concern is worth extending directly to this day's content as well: a ViT-based detection or tracking system (Section 9.1) carries exactly the same behavioral-profiling capability Day 31

§9.4 and Day 32 §9's ethics discussions developed for their own respective CNN-based systems — the architecture underlying a detector does not change what that detector's output can be used for. What is genuinely new here, worth flagging rather than glossing over, is scale: because a single pretrained ViT backbone (Section 9.3) is increasingly shared across many downstream applications rather than each application training its own separate model from scratch, a bias or vulnerability discovered in one widely-used pretrained checkpoint has a correspondingly wider blast radius than a comparable issue in a single, narrowly-deployed CNN would have — a genuine, architecture-scale-driven amplification of concerns this compendium has been developing incrementally since Day 26, worth carrying forward explicitly into Day 40 onward's vision-language content, where a shared visual encoder's influence extends even further, into systems that also reason in natural language about what they see.

10 Research Frontiers

10.1 Efficient ViT Variants for Edge Deployment

Section 5.3's parameter-count comparison showed ViT-Base is comparable in size to a large CNN, not dramatically larger — but Vision Transformers as a family have historically been more computationally expensive to run at inference time than an efficiency-optimized CNN (Day 30 §6's MobileNet/EfficientNet family specifically), motivating an active research direction adapting Day 30 §6.2's depthwise separable convolution efficiency principles, and related techniques, into transformer-native efficient architectures suitable for mobile and edge deployment under the same latency and power constraints Day 30 §6.1 originally motivated.

Approach	Core Idea	Compendium Connection
Hybrid conv-attention stems	Use convolution for early, local processing; attention only later	Day 39 §6's hybrid architectures
Reduced attention resolution	Compute full attention on a downsampled token grid	Directly addresses §2.7's quadratic cost
Efficient attention approximations	Replace exact softmax attention with a cheaper approximation	Day 39 §4's broader survey
Knowledge distillation to a smaller ViT	Train a compact ViT student from a larger ViT teacher	The same principle as §7.4's DeiT, applied for efficiency rather than data-hunger

10.2 Hierarchical and Windowed Attention

Section 3.4.1's worked example already previewed the core technical problem this research direction addresses directly: self-attention's quadratic cost in sequence length (Section 2.7) makes a plain ViT's single-resolution, full-image attention increasingly expensive as resolution grows or patch size shrinks. This is precisely Day 39 §2's central technical problem, and Day 39 §3's Swin Transformer — restricting

attention to local windows, then shifting those windows across layers to allow cross-window information flow — is this research direction's most influential concrete answer, developed there in full as this day's immediate successor topic.

10.3 Scaling Laws for Vision Transformers

Section 6.3's crossover point and Section 7.1's JFT-300M pretraining scale both raise a natural further question: how, precisely, does ViT performance continue to scale as model size, data size, and training compute all continue to grow well beyond what this day's examples have covered? Empirical scaling-law research — extending the general scaling-law framework Day 30 §10.1 introduced for vision models broadly — has found that Vision Transformers scale predictably and favorably with increased size and data, a large part of why ViT-Huge and even larger variants (Section 5.4) have become the default large-scale visual backbone choice underlying the vision-language systems Day 40 onward develops.

The general empirical pattern found in this research is worth stating in its qualitative form, since the precise numeric fits are specific to particular studies and datasets: error tends to decrease as a smooth, predictable power-law function of model size, dataset size, and training compute, over a wide range before eventually flattening out — meaning that, within this predictable range, a researcher can reasonably forecast how much additional accuracy a given increase in scale will purchase, before actually running the larger, more expensive experiment. This predictability is itself a genuinely useful practical property, distinct from Section 6.3's crossover point: the crossover point is about WHETHER ViT beats a CNN at a given scale; scaling laws are about HOW MUCH further improvement to expect from additional scale, once ViT has already established its advantage. Both together inform Section 5.4's configuration-family choice and Section 7.1's pretraining-scale decisions for any team with the resources to make them — though it is worth noting explicitly that this predictability holds empirically over the ranges these studies have actually tested, and extrapolating far beyond tested ranges remains a genuine open question, consistent with Section 10.7's broader caution against treating any of this day's content as fully settled.

10.4 Forward References

Day 39, immediately following this day, develops the architectural response to Section 10.2's windowed-attention preview in full: Swin Transformer's hierarchical, multi-resolution design; efficient attention mechanisms more broadly; DeiT's distillation approach (previewed already in Section 7.4); and ConvNeXt's "modernized CNN" alternative, revisiting Day 30 §8's brief mention with the full architectural detail this day's transformer-fundamentals content now makes possible to develop. Day 40 then turns to language, beginning this module's extension from vision alone into genuinely multimodal, vision-and-language systems — with Section 9.3's ViT-as-visual-encoder discussion as the direct technical bridge.

10.5 Closing Perspective

This day closed the loop Section 1 opened: having spent eleven days assuming convolution as the vision field's core operation, this day developed, mechanically and from first principles, exactly what changes when that operation is replaced with self-attention instead — how an image becomes a sequence (Section 3), how spatial information is restored to that sequence (Section 4), how the resulting architecture is assembled, trained, and used in practice (Sections 5, 7, and 8), and precisely why it behaves differently from a CNN under differing data availability (Section 6). Every remaining day in this five-day module builds directly on this foundation, whether refining the architecture itself (Day 39) or extending it into genuinely multimodal territory (Days 40 through 42) — this day is a beginning, not a self-contained conclusion.

10.6 Choosing Between CNN and ViT Today: A Practical Framework

It is worth closing with a synthesis in the same spirit as Day 30 §9's backbone-selection framework, now specifically for the CNN-versus-ViT choice this entire day has developed. None of the dimensions below should be considered in isolation — a real project typically faces constraints along more than one simultaneously, exactly as Day 30 §9 cautioned for backbone choice generally.

Constraint	Favors CNN	Favors ViT
Available labeled data	Small to moderate (§6.2–6.3)	Large, or large-scale pretrained checkpoint available (§7.5)
Compute/latency budget	Tight — efficient CNN families (Day 30 §6)	Generous, or efficient ViT variant (§10.1, Day 39)
Need for long-range relationships	Adequate — most natural-image tasks	High — tasks with genuinely non-local structure (§6.5)
Downstream dense prediction (detection/segmentation)	Mature, well-understood pipelines (Days 31, 33)	Hierarchical variant needed (Day 39 §3), not plain ViT
Need for a shared vision-language backbone	Not applicable	Standard choice (§9.3, Day 40 onward)
Interpretability requirements	Grad-CAM and friends (Day 37 §3–4)	Attention visualization available for free (§8.5), with faithfulness caveats (Day 37 §5.2)

As with every comparison this compendium has developed since Day 30 §9, there is no universal winner in this table — only a fit for a given project's actual constraints. What has changed, concretely, since Day 30 §9's original framework was first introduced is that the table now has a genuine second architecture family to weigh against the CNN lineage, not merely a single dominant default — and PP16's own Task A versus Task B comparison, developed in full in this day's assignment brief, is designed to make exactly this weighing process concrete on your own results rather than only abstract in this table's prose.

10.7 Open Research Questions

It is worth closing this day's theoretical content by naming several genuinely open questions this day's material does not fully resolve, rather than presenting the field as a settled matter — a posture consistent with this compendium's own repeated caution against declaring any single architecture a permanent, universal winner.

- **Is Section 6's inductive-bias tradeoff fundamental, or an artifact of current training techniques?** Section 7.4's DeiT result — narrowing the data-hunger gap substantially via distillation rather than raw data scale — suggests at least part of the gap may be addressable through better training methodology rather than being an immovable property of the architecture itself; how much further this can be pushed remains actively studied.
- **How should Section 6.3's crossover point be predicted in advance, for a new task and dataset, rather than only measured after the fact?** Section 10.3's scaling-law research offers partial answers for well-studied benchmark tasks, but a general, reliable predictive theory for an arbitrary new application remains an open practical question, not a solved one.
- **Will hierarchical, windowed architectures (Day 39 §3) or efficient global-attention approximations (Day 39 §4) ultimately prove the more durable solution to Section 2.7's quadratic cost problem?** Both remain active research directions, per Section 10.2, without a settled consensus favoring one over the other for every use case.
- **How much of Section 9.3's vision-language ViT-as-encoder pattern will persist as vision-language architectures continue to evolve?** Day 40 through 42 develop the current state of this question in detail, but it is worth entering that material aware that today's standard pattern is a snapshot of an actively evolving field, not a permanent architectural conclusion.

11 Common Pitfalls: A Practical Debugging Checklist

This day combines a genuinely new architectural mechanism with data-scale considerations that differ substantially from every earlier day's CNN-focused content. Because the pitfalls span from low-level tensor-shape mistakes through to higher-level conceptual misreadings of Section 6's argument, they are organized here into three tiers — architectural, training, and interpretive — worth working through in roughly that order.

Architectural Pitfalls

- **Verify patch size divides image size evenly before debugging anything else.** Per §3.4.1 and §8.4, a mismatched patch/image size produces an ill-defined patch grid — check this first, since it is the single most common from-scratch implementation error.
- **Confirm the [CLS] token is correctly implemented and correctly extracted for classification.** Per §4.3 and §8.4, both prepending the token and correctly selecting its final representation (not

some other token's, and not an average across all tokens) are required for the architecture to work as intended.

- **Double-check the scaling factor in your attention implementation.** Per §2.4, omitting the $\sqrt{d_k}$ division is a subtle bug that will not raise an error but can measurably degrade training stability, particularly at larger embedding dimensions — verify this line explicitly in a from-scratch implementation.
- **Confirm residual connections wrap both the attention and feed-forward sub-layers.** Per §2.5, an encoder block missing either residual path loses the gradient-flow benefit Day 30 §3.3–§3.4 established was necessary for trainable depth — a missing residual connection can produce a network that trains but plateaus well below expected accuracy, with no explicit error to flag the cause.

Training Pitfalls

- **Do not interpret Task A's lower accuracy as an implementation bug before checking Section 6's argument.** Per §6.2's Research Spotlight, a from-scratch ViT trained on a compact dataset is EXPECTED to underperform a fine-tuned pretrained ViT — this is the assignment's intended empirical result, not evidence of a mistake.
- **Check for a missing or misconfigured learning rate warmup phase.** Per §7.6, transformers are considerably more sensitive to an overly large early learning rate than a typical CNN — early-training divergence or oscillation should prompt a check of the warmup schedule before any other hyperparameter.
- **Watch for underestimating how many training steps Task A needs before results become meaningful.** Per §7's data-hunger discussion, a from-scratch ViT needs more training signal than a comparable CNN to show sensible learning curves at all — a seemingly stalled Task A training run may simply need more steps, not a different learning rate or architecture.
- **Consider linear probing as a diagnostic step before committing to full fine-tuning.** Per §8.2, if full fine-tuning produces surprisingly poor Task B results on a very small dataset, compare against a linear-probing baseline (frozen encoder, trained head only) to check whether full fine-tuning is overwriting useful pretrained structure.

A Fourth Tier: Comparison-Specific Pitfalls

Because PP16's core deliverable is a direct comparison between Task A and Task B rather than either task in isolation, a fourth tier of pitfalls specific to that comparison is worth naming separately from the architectural, training, and interpretive tiers above.

- **Ensure both tasks are evaluated on an identical held-out test set.** A comparison where Task A and Task B were evaluated on even slightly different data splits is not a valid comparison at all,

regardless of how carefully each task was otherwise implemented — verify the exact same test indices are used for both.

- **Report training time under comparable conditions.** Comparing Task A's training time on a T4 against Task B's training time on an A100 (or vice versa) conflates a hardware difference with a genuine architectural difference — run both tasks on identical hardware for any timing comparison to be meaningful.
- **Distinguish wall-clock time from number of training steps.** Task B's fine-tuning may reach strong accuracy in fewer total steps than Task A, but if each Task B step is itself more expensive (a larger pretrained model, per §5.3–5.4), wall-clock time and step count can tell different stories — report both where feasible, and be explicit about which one your written reflection is discussing at each point.
- **Do not average away a meaningful difference between multiple runs without reporting the variance.** Per §11's reproducibility note, a from-scratch ViT's run-to-run variance can be genuinely larger than a fine-tuned model's — reporting only a single run's numbers for both tasks can make a real difference look smaller or larger than it consistently is.

Interpretive Pitfalls

- **Check position embedding handling explicitly whenever input resolution changes.** Per §8.3, fine-tuning or evaluating at a different resolution than pretraining requires correctly interpolating learned position embeddings — a silent mismatch here degrades performance without raising an error.
- **Verify preprocessing normalization matches the pretrained checkpoint's own training statistics.** Per §8.4 and Day 27 §5.2's original warning, this is a recurring silent failure mode across this entire compendium, and applies to ViT checkpoints exactly as it applies to CNN backbones.
- **Do not treat a single attention map as a fully faithful explanation.** Per §8.5 and Day 37 §5.2's faithfulness caveat, cross-check a surprising or high-stakes attention-based conclusion against an independent method before relying on it.

A Note on Reproducibility

Vision Transformer training shares every randomness source this compendium has flagged for CNN training (weight initialization, augmentation, per Day 29 §8.2), plus one worth flagging specifically here: because a from-scratch ViT (PP16 Task A) has no built-in inductive bias to constrain its learned spatial relevance patterns (§6.1), different random initializations can, in principle, discover meaningfully different attention patterns even when trained on identical data — a source of run-to-run variance with no comparably strong analogue in a CNN, whose architecture already constrains the space of learnable spatial patterns

considerably more tightly. Fixing random seeds explicitly, and reporting results as an average across a small number of runs where feasible, is worth doing for any comparison drawn from Task A specifically.

A Note on Comparing Against Published Benchmarks

Published ViT benchmark results — including the original paper's own headline figures — were obtained using JFT-300M-scale pretraining (§7.1) or, at minimum, ImageNet-21k-scale pretraining, far beyond what a single Colab session's Task A can reproduce. A PP16 Task A result well below published from-scratch ViT figures is the expected, unsurprising outcome given this scale difference, not necessarily evidence of an implementation problem — the meaningful comparison, exactly as this compendium's every other graded assignment has established (Day 31 §8.3, Day 34 §9.5, Day 37 §11), is PP16's own Task A against its own Task B under your matched conditions.

A Complete Diagnostic Walkthrough

It is worth working through one complete, realistic diagnostic scenario end to end, tying together several of this checklist's individual items into the kind of systematic debugging process a real implementation session actually requires — echoing the diagnostic-walkthrough format Day 37 §11 established for its own domain.

Worked Example: Diagnosing a Stalled Task A Training Run

Symptom: PP16's Task A loss decreases very slowly for the first several hundred steps, then appears to plateau at a level barely better than random guessing, while Task B (trained in the same session, same codebase) converges normally.

Step 1 — rule out an architectural bug first, per this checklist's architectural tier: verify patch embedding output shape, [CLS] token presence, and encoder block shape-preservation on a small toy input (§8.6). Suppose these all check out correctly.

Step 2 — check the learning rate warmup schedule specifically, per this checklist's training tier and §7.6: suppose the training script omitted warmup entirely, applying the full target learning rate from step one. A transformer's attention softmax (§2.4) is specifically vulnerable to this, in a way a comparable CNN training run often tolerates without incident.

Step 3 — add a linear warmup phase over the first 5–10% of training steps (§7.6, Figure 15) and re-run. Suppose the loss now decreases smoothly and reaches a reasonable, if still Task-B-lagging, final accuracy.

Step 4 — before concluding the fix is complete, revisit §6.2's expected outcome: Task A's final accuracy, even after fixing the warmup bug, should still measurably lag Task B's — if it does not, that itself would be worth investigating, since it would depart from this day's central predicted pattern rather than confirm it.

This walkthrough illustrates the checklist's intended use directly: work through the tiers in order (architectural, training, interpretive), fix one plausible cause at a time, and re-evaluate against §6's expected pattern rather than an assumption that any remaining gap must indicate a further bug.

Glossary of Terms

This day introduced a substantial amount of transformer-specific vocabulary alongside vision-specific applications of it. The table below collects the most important terms in one place, each with a pointer back to the section where it was developed in full.

Term	Definition
Transformer	The general attention-based architecture family; not vision-specific in itself (§1.6)
Self-attention	A mechanism computing a weighted combination of every sequence element from every other element (§2.2)
Query, key, value	The three learned linear projections of a token embedding that self-attention combines (§2.2)
Multi-head attention	Running several independent attention computations in parallel and combining their outputs (§2.3)
Scaled dot-product attention	Attention scores divided by $\sqrt{d_k}$ before softmax, avoiding a vanishing-gradient regime (§2.4)
Layer normalization	Normalizing each sequence element's representation across its own feature dimension (§2.5)
Residual connection (transformer)	An additive skip path around attention/feed-forward sub-layers, preserving gradient flow (§2.5)
Pre-norm / post-norm	Whether layer normalization is applied before or after each sub-layer (§2.5)
Causal vs. bidirectional attention	Whether a token may attend only to earlier tokens, or to the full sequence (§2.8)
Patch embedding	Dividing an image into fixed patches, flattening, and linearly projecting each into a token (§3.1–3.2)
Position embedding	A learned or fixed vector added to each token to restore spatial ordering information (§4.1–4.2)
Relative position encoding	Encoding a token pair's relative distance directly within the attention computation (§4.5)
[CLS] token	A prepended learnable token whose final representation summarizes the whole image (§4.3)
Inductive bias	An architectural assumption (e.g. CNN locality) that narrows the learnable function space before training (§6.1)
Translation equivariance	A shifted input produces a correspondingly shifted output — distinct from invariance (§6.4)
Hierarchical compositionality	The implicit bias of stacked layers building simple patterns into progressively complex ones (§6.4)
Data-hunger crossover point	The training-data scale beyond which ViT matches or exceeds a comparable CNN (§6.3)
JFT-300M	The ~300-million-image dataset used for ViT's original massive-scale pretraining (§7.1)

Term	Definition
Stochastic depth	Randomly dropping entire encoder blocks during training as a regularization technique (§7.3)
DeiT	Data-efficient image Transformers — using CNN-teacher knowledge distillation to reduce ViT's data requirement (§7.4)
Linear probing	Fine-tuning only a new classification head while freezing the pretrained encoder (§8.2)
Learning rate warmup	Gradually increasing the learning rate over early training steps before decay begins (§7.6)
Mixed-precision training	Performing most training computation in reduced numeric precision for speed and memory savings (§7.7)
ViT-Small / Base / Large / Huge	Standard named ViT configurations differing in encoder depth and width (§5.4)
Position embedding interpolation	Resampling pretrained position embeddings onto a new patch grid at a different resolution (§8.3)
Gradient accumulation	Summing gradients across several smaller micro-batches before a single weight update, to fit memory limits (§8.7)

Chapter Summary: Key Takeaways

- **A Vision Transformer replaces convolution with self-attention as its core operation (§1–2)** — a genuine architectural departure, not an incremental refinement of the CNN lineage Day 30 traced, and one that first proved itself in language (§1.2) five years before vision adopted it (§1.3).
- **Self-attention computes query/key/value projections and a scaled, softmax-normalized weighted combination (§2.2–2.4)** — a mechanism worth having concretely worked through by hand (§2.2.1), since every later section builds on this exact computation.
- **Patch embedding converts an image into a sequence (§3)**, flattening and linearly projecting fixed-size patches — mathematically equivalent to a single strided convolution (§3.2) — with patch size directly and quadratically determining attention cost (§3.4.1).
- **Position embeddings and the [CLS] token restore spatial ordering and provide a summary representation (§4)**, both compensating for properties self-attention does not provide automatically — and learned position embeddings empirically discover an implicit approximation of 2D structure purely from training (§4.2).
- **Stacking transformer encoder blocks and reading out the final [CLS] token completes the architecture (§5)**, with a parameter count comparable to, not dramatically larger than, an equivalent CNN — traced completely in a full numeric forward-pass worked example (§5.5).

- **ViT's lack of built-in inductive bias is the mechanistic reason it is more data-hungry than a CNN (§6)** — the conceptual core of this entire day, extending beyond locality alone to compositionality and equivariance (§6.4), with a genuine cost on the CNN side of the ledger too (§6.5).
- **Massive-scale pretraining, strong augmentation, extra regularization, distillation, and careful optimizer/schedule choices (§7)** are five distinct practical responses to the data-hunger problem and ViT's own training sensitivities; transfer learning remains the practical default for most applications (§7.5).
- **A pretrained ViT is now the standard visual encoder inside vision-language models (§9.3)**, and reaches across medical imaging, satellite imagery, and video as well (§9.4) — the direct, load-bearing connection into the remainder of this five-day module.
- **Efficient attention and hierarchical architectures (§10)** are the direct research response to self-attention's quadratic cost — developed in full starting immediately in Day 39.

Assignment / Practical Project Brief

PP16: Vision Transformer Implementation

★ GRADED ASSIGNMENT

This is a graded assignment, carrying a formal rubric (Section D). The core requirement: implement both a from-scratch Vision Transformer (Task A) and a fine-tuned pretrained Vision Transformer (Task B), and directly compare them — the comparison is what demonstrates Section 6's inductive-bias argument empirically, and is a required deliverable, not an optional extension.

A Core Requirements

Task A — From-Scratch ViT. Implement patch embedding (§3), position embeddings (§4.1–4.2), the [CLS] token (§4.3), and a small transformer encoder stack (4–6 layers is sufficient — §5.1's full ViT-Base depth is not required here) entirely from first principles, without using a pre-built ViT implementation for this task specifically. Train on a compact classification dataset (CIFAR-10/100 or a Tiny-ImageNet subset are all reasonable). The goal of this task is demonstrating mechanism understanding, not achieving state-of-the-art accuracy — a short training run producing a working, learning model is sufficient.

Task B — Fine-Tune a Pretrained ViT. Load a pretrained checkpoint (`google/vit-base-patch16-224` via Hugging Face `transformers` is the recommended default, per §8.1) and fine-tune it on the same dataset used for Task A, or a different dataset if you prefer — either choice is acceptable provided your written comparison accounts for it. This task directly exercises the transfer-learning workflow developed in §7.5 and §8.2.

B Dataset

A compact classification dataset, sized for tractable training within a single working session, is required for Task A specifically (Task B's fine-tuning will typically converge faster on the identical dataset, per §7's data-hunger discussion).

Dataset Option	Size	Note
CIFAR-10	60,000 images, 10 classes	Smallest, fastest option; images are only 32×32 , requiring patch size adjustment (§3.4.1's worked example logic applies directly)
CIFAR-100	60,000 images, 100 classes	Same image size as CIFAR-10, more classes — a harder Task A target
Tiny-ImageNet subset	A few thousand images, a handful of classes	Larger, more natural-image-like; closer to Task B's own pretraining domain

A worked sizing note for the small-image datasets specifically: CIFAR-10/100's 32×32 images, at a standard 16×16 patch size, would produce only a $2 \times 2 = 4$ -patch sequence (5 tokens including [CLS], per §4.3) — far too short a sequence for Section 2's self-attention mechanism to do anything structurally meaningful with, and not representative of the patch-count scale (§3.4.1's 196-token example) this day's content has developed throughout. A smaller patch size — 4×4 (producing 64 patches) or 8×8 (producing 16 patches) — is the appropriate adjustment for Task A specifically on these datasets, following exactly the same patch-size-as-a-tunable-hyperparameter reasoning §3.4 developed in full.

C Deliverables

- Task A: a working from-scratch ViT implementation (patch embedding, position embeddings, [CLS] token, transformer encoder stack), trained and evaluated on your chosen dataset.
- Task B: a fine-tuned pretrained ViT, trained and evaluated on the same (or a clearly documented different) dataset.
- A direct comparison: final accuracy, total training time, and a qualitative discussion of data efficiency — how many training steps or epochs each task needed before showing meaningful learning, per §11's checklist.
- A short written reflection connecting your own measured results back to §6's inductive-bias argument — does your comparison show the expected pattern, and if not, what do you think explains the difference?
- Optional but encouraged: an attention visualization (§8.5) for at least one Task A and one Task B prediction, offering a direct, qualitative window into what each model actually learned to attend to.

A brief guide to interpreting your own results, worth reading before concluding your comparison shows an implementation problem: Task A's accuracy on a compact dataset trained for a short session is expected to be modest — meaningfully above random chance (demonstrating the architecture is genuinely learning something, per §11's checklist), but well below Task B's fine-tuned result. Task B, by contrast, should reach a comparatively strong accuracy relatively quickly, often within a small number of epochs, precisely because it starts from weights already shaped by large-scale pretraining (§7.1, §7.5). A large gap between the two is the CORRECT, expected outcome this assignment is designed to produce — document it, explain it via §6's argument, and resist the temptation to over-train Task A indefinitely chasing a smaller gap than the underlying data-hunger phenomenon actually predicts at this dataset scale.

A further note on scope: this assignment deliberately does not require matching or approaching any published state-of-the-art accuracy figure for your chosen dataset, for either task. Task A's purpose (§D) is demonstrating correct mechanism implementation, not competitive accuracy; Task B's purpose is demonstrating a correct transfer-learning workflow, not exhaustive hyperparameter tuning toward a leaderboard result. A submission that clearly demonstrates both correct implementations and a well-reasoned comparison, even at modest absolute accuracy for both tasks, will be graded more favorably than a submission that reports impressive numbers without a rigorous, honestly-reasoned comparison behind them — consistent with §D's own weighting, where the comparison and reflection component (30%) meaningfully outweighs either task's raw implementation credit taken alone.

D Grading Rubric

Rubric Component	Weight	What's Assessed
Task A implementation	35%	Correct patch embedding, position embeddings, [CLS] token, and encoder stack, implemented from first principles
Task B implementation	20%	Correct fine-tuning workflow; sensible training curve
Comparison & written reflection	30%	Rigorous accuracy/time/data-efficiency comparison; reflection genuinely engages with §6's argument using your own results
Code quality & write-up	15%	Clean, documented code; clear explanation of architectural choices

Note the weighting deliberately: Task A's from-scratch implementation carries the largest single share, since it is the task that actually demonstrates you can build the mechanism Sections 3 through 5 developed, rather than only knowing how to call a pretrained model. The comparison and reflection together carry nearly a third of the grade — a technically correct Task A and Task B with a superficial or missing comparison will not score well.

A more granular breakdown of Task A's 35%, since it is this assignment's single largest component: roughly a third of this share rewards correct patch embedding mechanics (§3.1–3.2, including correct handling of your chosen dataset's image size per §3.4.1's worked example), a third rewards correct position embedding and [CLS] token implementation (§4.1–4.3), and the remaining third rewards a correctly assembled encoder stack (§2.5, §5.1) with working residual connections and a functioning classification head (§5.2) — a submission strong in one area but clearly incomplete in another should expect a proportionally partial score on this component, not a pass/fail judgment on the task as a whole.

E Tips and Common Pitfalls

- **Adjust patch size for small images.** CIFAR-10/100's 32×32 images do not divide sensibly into the standard 16×16 patch size (per §3.4.1's worked example, this would produce only a $2\times 2 = 4$ -token sequence) — a smaller patch size (4×4 or 8×8) is more appropriate for this scale, per this brief's own §B sizing note above.
- **Do not skip the [CLS] token in Task A.** Per §4.3 and §8.4, both prepending it and correctly using its final representation for classification are required — an easy detail to omit when implementing from scratch.
- **Budget realistic expectations for Task A's accuracy.** Per §6's entire argument, a from-scratch ViT on a compact dataset is not expected to match Task B's fine-tuned result — a meaningfully lower Task A accuracy is the expected, correct outcome your written reflection should discuss, not a result to be troubleshot away.
- **Runtime guidance: target an A100 GPU via Colab Pay-As-You-Go where available, but design for graceful T4 fallback.** Both tasks are comfortably lightweight for an A100, but A100 allocation is not guaranteed every session even on paid tiers — detect available GPU at runtime and reduce Task A's batch size and epoch count automatically on a T4 rather than failing outright, using §8.7's gradient accumulation technique if a smaller batch size alone measurably hurts training stability.
- **Match preprocessing to Task B's pretrained checkpoint exactly.** Per §8.4 and Day 27 §5.2, use the checkpoint's own documented normalization statistics, not a default or assumed set of values.
- **Verify each architectural component on a toy input before full training.** Per §8.6, checking patch embedding output shape, encoder block shape-preservation, and [CLS] token extraction on a small synthetic input catches the majority of Task A implementation bugs far more cheaply than debugging after a full training run.
- **Include a learning rate warmup phase in both tasks.** Per §7.6, omitting warmup is a common and easily-diagnosed cause of early training instability specifically for transformer architectures — verify this explicitly if either task's loss curve looks unstable in its first steps.

- **Keep Task A and Task B's evaluation protocol identical.** Use the same train/validation split, the same evaluation metric computation, and the same held-out test set for both tasks — an inconsistent evaluation protocol between the two tasks invalidates the direct comparison this assignment's grading weights so heavily (§D).

F Submission Checklist

- Task A: from-scratch ViT implemented (patch embedding, position embeddings, [CLS] token, encoder stack) and trained
- Task A: patch size appropriately adjusted for your chosen dataset's image size (§B)
- Task A: individual components verified on a toy input before full training (§8.6, §E)
- Task B: pretrained ViT fine-tuned on the chosen dataset, with preprocessing matched to the checkpoint's own documented statistics
- Runtime detects available GPU and falls back gracefully from A100 to T4 configuration
- Comparison table completed with real measured accuracy, training time, and data-efficiency observations, not placeholders
- Evaluation protocol (train/val split, metric computation) kept identical between Task A and Task B
- Written reflection explicitly connects results to §6's inductive-bias argument, engaging honestly with any results that diverge from the expected pattern
- Code is documented, and a short README explains how to reproduce your results

G Extension Ideas (Optional)

The following extensions are not required for a complete PP16 submission, but are worth attempting for students specifically interested in pushing this day's content further — none are graded, and none should be attempted at the expense of Sections A through F's required core.

- **Implement DeiT-style distillation (§7.4)** as a third variant of Task A, training a from-scratch ViT with a CNN teacher's guidance, and comparing its data efficiency against both the plain Task A and the fine-tuned Task B.
- **Visualize attention maps (§8.5)** for both Task A and Task B on the same held-out example images, and compare qualitatively what each model appears to have learned to attend to.
- **Experiment with §4.4's 2D-aware or relative position embeddings** in place of Task A's standard learned absolute position embeddings, and check whether this measurably changes Task A's data efficiency on your chosen dataset.

- **Benchmark Task A's actual training memory footprint** at a few different patch sizes, directly verifying §3.4.1's quadratic attention-cost scaling on your own hardware rather than taking the worked example's numbers on faith.

End of Day 38 — first day of the Vision Transformers & Vision-Language Models module. Ready for gap review and expansion into full compendium.